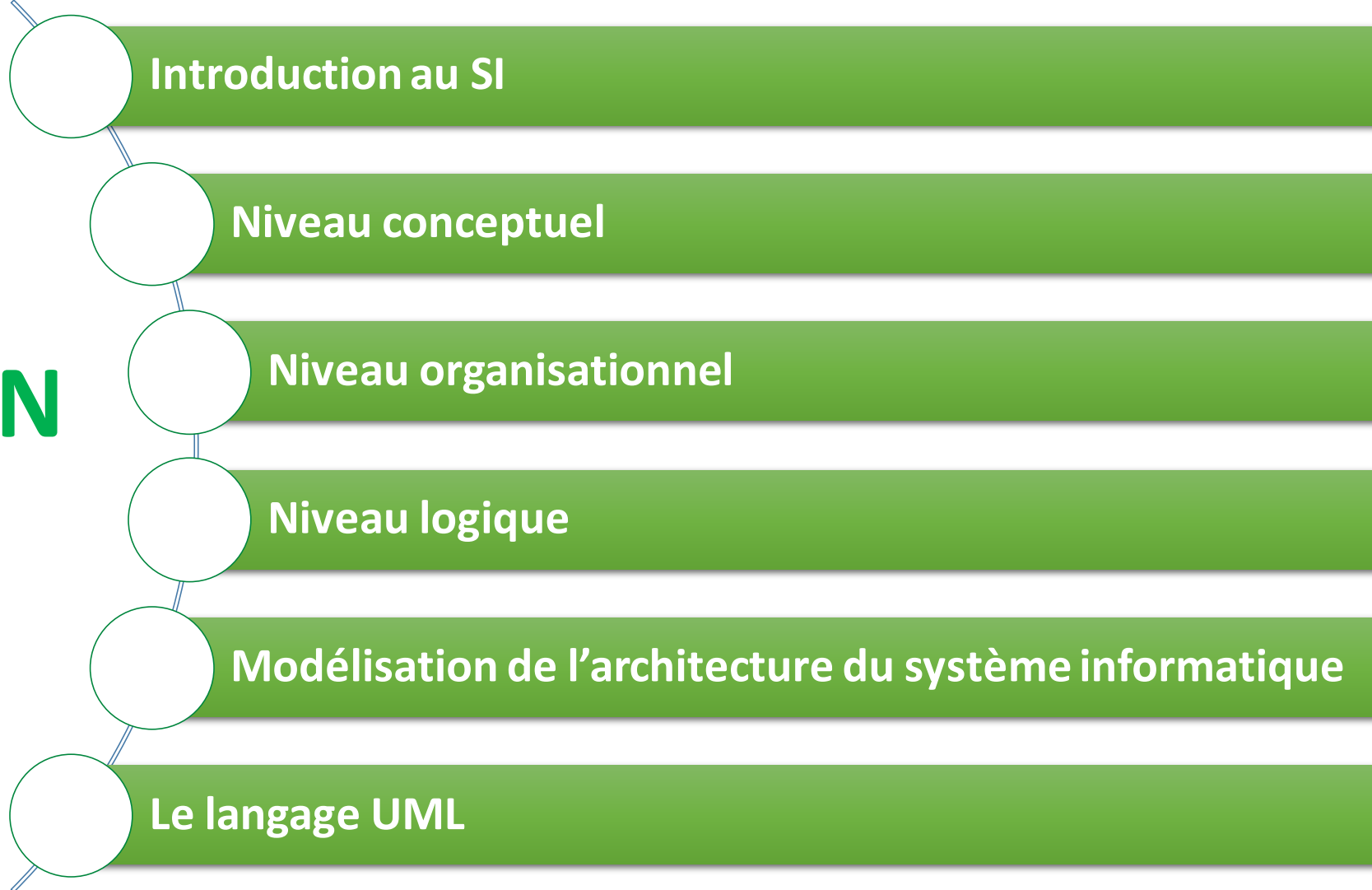


Conception des systèmes d'information

Filière 3^{ème} Année Ingénierie Informatique & Réseaux

Akram CHHAYBI
A.Chhaybi@emsi.ma

PLAN



I- Introduction aux Systèmes d'information (SI)

1-Présentation de la méthode MERISE

Dans une entreprise moderne, on manipule chaque jour une grande quantité d'informations :



Finance
(factures, paiements, budgets)



(commandes, réclamations, fidélisation)



Ressources humaines
(employés, contrats, salaires)

**Sans organisation, ces informations seraient dispersées,
redondantes ou incohérentes.**

I- Introduction au SI

I- Introduction aux Systèmes d'information (SI)

Pour que le SI soit efficace, les informations doivent être **structurées**, **accessibles** et **stockées** de manière **optimale**.

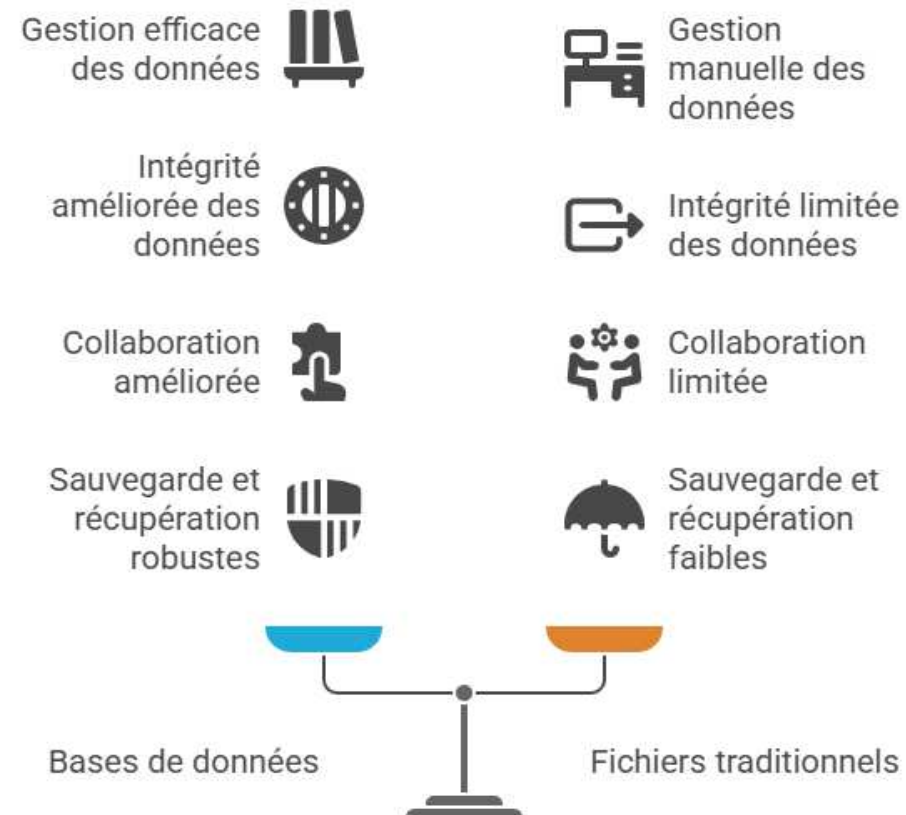


Base de données

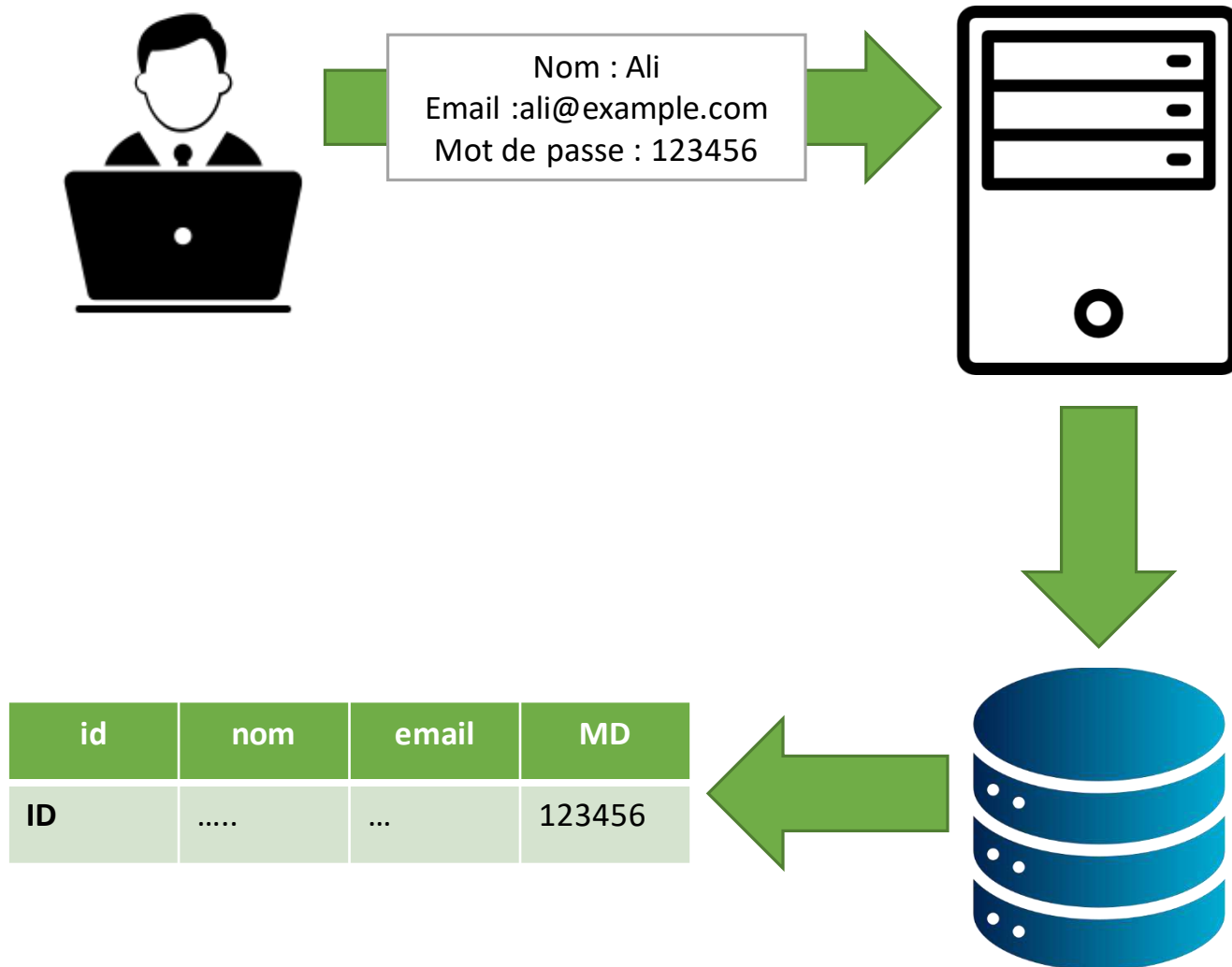
Une base de données est un système informatique pour **collecter**, **organiser** et **stocker** des informations de manière **structurée** et **électronique**, **permettant** leur gestion, leur consultation et leur mise à jour. Elle fonctionne comme un classeur numérique organisé où les données (textes, chiffres, images, etc.) sont stockées dans des **structures**, souvent des **tables avec des lignes et des colonnes**, pour un accès efficace. **Un système de gestion de base de données (SGBD)** est le logiciel utilisé pour interagir avec ces données.

I- Introduction aux Systèmes d'information (SI)

Avantages des Bases de Données (BD) vs. Fichiers traditionnels:



I- Introduction aux Systèmes d'information (SI)



- Le serveur reçoit la requête.
- Il vérifie la validité des données (ex. email valide, mot de passe assez long).
- Ensuite, il prépare une **requête SQL**

- Le SGBD (ex. MySQL, PostgreSQL) exécute la requête.
- Les données de l'utilisateur sont stockées dans la table utilisateurs.

I- Introduction aux Systèmes d'information (SI)

Construire une base de données ne se fait pas au hasard. Il faut analyser les besoins, modéliser les données et structurer le système avant de l'implémenter. C'est exactement ce que propose la méthode Merise :

Définition de la méthode MERISE:

La méthode Merise est une méthode de conception des systèmes d'information qui permet de modéliser et d'organiser les données et les traitements d'une organisation. Elle repose sur une approche par niveaux successifs :

1. **Niveau Conceptuel:** Décrire les besoins métiers de manière générale, sans technologie → Quoi ? Pourquoi
2. **Niveau Organisationnel:** Préciser les règles de gestion, l'organisation et les acteurs → Qui ? Quand ? Où ?
3. **Niveau Logique (ou Technologique/Physique):** Mettre en œuvre techniquement le système (base de données, programmes) → Comment ?

I- Introduction aux Systèmes d'information (SI)

Niveau Conceptuel

C'est la première étape de la méthode Merise, l'objectif c'est de décrire le besoin métier sans se soucier de la technique.
On répond à :

- **Quoi ?** → **quelles données ?**
- **Pourquoi ?** → **à quoi elles servent ?**

Résultat : une vision claire du système d'information idéal vu par l'entreprise.

Exemple : Gestion des commandes

Dans une petite boutique en ligne, **les clients passent des commandes** pour différents produits.

- Chaque **client** peut passer plusieurs **commandes**.
- Chaque **commande** peut contenir plusieurs **produits**.
- Chaque **produit** peut apparaître dans plusieurs **commandes**

**On représente les entités et leurs relations sans parler encore de base de données.
Cela permet de comprendre le métier avant la conception technique.**

I- Introduction aux Systèmes d'information (SI)

Niveau Conceptuel

Dans une petite boutique en ligne, **les clients passent des commandes** pour différents produits.

- Chaque **client** peut passer plusieurs **commandes**.
- Chaque **commande** peut contenir plusieurs **produits**.
- Chaque **produit** peut apparaître dans plusieurs **commandes**

Entités :

- **Client** : NumClient, Nom, Email
- **Commande** : NumCommande, Date, Montant
- **Produit** : RefProduit, NomProduit, Prix.

Relations :

- **Client** peut passer plusieurs commandes.
- **Commande** peut contenir plusieurs produits **Produit**.
- **Produit** peut être dans plusieurs commandes.

I- Introduction aux Systèmes d'information (SI)

Niveau Organisationnel

- C'est la traduction du niveau conceptuel en un modèle adapté à **un SGBD relationnel**.
- **Objectif** : Préparer la base de données avant l'implémentation technique.
- On répond aux questions :
 - **Comment organiser les données en tables ?**
 - **Quelles clés primaires et étrangères utiliser ?**

Résultat : un Modèle prêt à être transformé en base physique.

Principaux éléments du MLD :

- ✓ Tables correspondant aux entités
- ✓ Clés primaires (PK) et clés étrangères (FK)
- ✓ Relations 1,N et N,M transformées en tables associatives

I- Introduction aux Systèmes d'information (SI)

Niveau Organisationnel

Exemple de Niveau Organisationnel

À partir du MCD Client-Commande-Produit :

Tables :

1. Client

- NumClient (PK)
- Nom
- Email

2. Commande

- NumCommande (PK)
- Date
- Montant
- NumClient (FK → Client)

3. Produit

- RefProduit (PK)
- NomProduit
- Prix

4. Commande_Produit (table associative pour la relation N,M)

- NumCommande (FK → Commande)
- RefProduit (FK → Produit)
- Quantité

I- Introduction aux Systèmes d'information (SI)

Niveau Physique

- C'est la mise en œuvre technique du modèle logique dans un **SGBD** concret, et son objectif est de créer la base de données réelle, prête à être utilisée par le système.
- On répond à la question : « Comment stocker et gérer les données ? »

Travaux typiques :

- ❖ Créer les tables physiques dans un SGBD (MySQL, Oracle, PostgreSQL...)
- ❖ Définir types de données pour chaque attribut (INT, VARCHAR, DATE...)
- ❖ Créer index, contraintes, clés primaires et étrangères
- ❖ Optimiser stockage et performance

Résumé : le niveau physique est la traduction finale du niveau organisationnel pour que le SI fonctionne concrètement.

- Chaque table du MLD devient une **table physique** avec ses colonnes et types de données.
- Les relations sont matérialisées par **clés étrangères**.
- La table associative **Commande_Produit** gère la relation **N,M**.
- Le système est maintenant prêt à **stocker et gérer les données** réellement.

I- Introduction aux Systèmes d'information (SI)

Niveau Physique

Langage de base de données:

On utilise un **langage spécialisé** pour créer, manipuler et interroger la base :

- SQL pour les bases relationnelles
- Langage spécifique aux NoSQL (JSON, requêtes MongoDB, etc.)

Pour les bases de données relationnelles, on utilise SQL (Structured Query Language), reconnu par tous les SGBD relationnels comme **MySQL, PostgreSQL, Oracle, SQL Server**.

SQL permet de :

- ❖ Créer les tables : CREATE TABLE
- ❖ Définir les relations : clés primaires et étrangères
- ❖ Insérer, modifier et interroger les données : INSERT, UPDATE, SELECT

II- Niveau conceptuel

II- Niveau Conceptuel

1. Introduction au Niveau Conceptuel : Définition et Rôle

Définition: Le niveau conceptuel est la première étape fondamentale de la conception d'un système d'information (SI) ou d'une base de données (BDD). Il vise à décrire les **données nécessaires** à l'organisation et les **règles de gestion** qui les régissent, et ce, de manière **complète, non redondante et indépendante** de toute contrainte technique logicielle ou matérielle.

Pour décrire les **Données** au niveau conceptuel est le **Modèle Conceptuel de Données (MCD)**, basé sur le formalisme **Entité-Association (E/A)**.

A. L'Entité (ou Type d'Entité):

L'entité est la représentation d'un objet ou d'un concept du monde réel qui a une existence propre et qui doit être mémorisé.

- **Règles de Création** : Une entité doit être identifiable.
- **Composants** :
 - **Nom** : Généralement un nom commun au singulier (ex: Client, Produit).
 - **Attributs** : Les propriétés qui décrivent l'entité (ex: Nom, Adresse).
 - **Identifiant** : L'attribut (ou ensemble d'attributs) qui garantit l'unicité de chaque occurrence (ex: idClient).

II- Niveau Conceptuel

B. L'Association (ou Type d'Association/Relation): L'association représente le lien sémantique ou l'interaction qui existe entre deux ou plusieurs entités.

Règles de Création : L'association est toujours nommée par un verbe à l'infinitif ou une action (ex: PASSER, CONTENIR).

Attributs d'Association : Certains attributs ne décrivent ni l'une ni l'autre des entités, mais le lien lui-même (ex: l'attribut **Quantité** pour l'association CONTENIR entre Commande et Produit).

C. Les Cardinalités (ou Multiplicités): Les cardinalités sont l'expression formelle des règles de gestion de l'organisation. Elles sont placées aux extrémités des associations et définissent la contrainte numérique du lien.

- **Format :** Couple (**Min, Max**), indiquant le nombre minimum et maximum de fois où une occurrence de l'entité participe à l'association.

Minimum (Min) :

- **0** : Participation **optionnelle**.
- **1** : Participation **obligatoire**.

Maximum (Max) :

- **1** : Participation **unique**.
- **N** (ou *****) : Participation **multiple** (plusieurs fois).

II- Niveau Conceptuel

Exemple:

Une Entreprise est responsable de la gestion de ses Projets. Il est entendu qu'un projet ne peut être géré que par une seule entreprise, même si celle-ci peut en piloter plusieurs. De son côté, chaque Projet nécessite l'utilisation d'Équipements pour sa réalisation. Un équipement donné ne peut être affecté qu'à un seul projet à la fois, mais peut aussi être non affecté. De plus, un projet doit impérativement avoir au moins un équipement pour être actif.

1. Étape Collaborative : Identification des Entités (Le "QUOI")

Les entités représentent les objets ou concepts dont le système doit conserver des informations. Elles sont caractérisées par leurs attributs, dont un identifiant unique (souligné).

Entité	Explication	Attributs (Identifiant souligné)
ENTREPRISE	L'acteur qui gère les projets.	idEntreprise, Nom, Adresse
PROJET	L'objet de travail planifié.	idProjet, Nom, DateDébut
ÉQUIPEMENT	Les ressources matérielles utilisées.	idEquipelement, Type, DateAcquisition

II- Niveau Conceptuel

2. Associations et Cardinalités (Les Règles du Jeu)

Les associations formalisent les liens logiques entre les entités, et les cardinalités traduisent les contraintes numériques définies dans le texte.

A. Analyse de l'Association 1 : GÉRER (Entre ENTREPRISE et PROJET)

Règles en Jeu :

1. Un projet doit être géré par **une seule** entreprise.
2. Une entreprise peut gérer **plusieurs** projets (ou aucun).

Question Posée	Règle de Gestion (Min, Max)	Explication
Côté ENTREPRISE : Combien de projet je peux manager?	Zéro (Min) et Un (Max)	(0, N) : C'est optionnel (l'entreprise peut être nouvelle) et multiple (elle gère plusieurs projets).
Côté PROJET : Si je prends une ENTREPRISE, combien de Projets gère-t-elle ?	Un (Min) ou Plusieurs (Max)	(1, 1) : C'est obligatoire et unique (un seul manager pour le projet).

II- Niveau Conceptuel

Alors:

ENTREPRISE $\underline{(0, N)}$ GÉRER $\underline{(1, 1)}$ PROJET

B. Analyse de l'Association 2 : UTILISER (Entre PROJET et ÉQUIPEMENT)

Règles en Jeu :

1. Un **Équipement** est affecté à **un seul** projet à la fois (ou aucun, s'il est au dépôt).
2. Un **Projet** doit utiliser **au moins un** équipement pour démarrer, et il peut en utiliser plusieurs.

PROJET $\underline{(1, N)}$ UTILISER $\underline{(0, 1)}$ ÉQUIPEMENT

II- Niveau Conceptuel

Exemple à faire:

La Bibliothèque Municipale souhaite informatiser une partie de son fonctionnement pour mieux gérer les livres et les emprunts de ses abonnés. Chaque abonné possède un numéro unique, un nom et un prénom. Les abonnés peuvent être des étudiants ou des professeurs. Chaque livre est identifié par un numéro ISBN unique. Un livre possède un titre, un ou plusieurs auteurs, et une année de publication. Un auteur est identifié par un code unique, un nom et un prénom. Un livre doit avoir au moins un auteur, et un auteur peut écrire plusieurs livres. Pour emprunter, un abonné peut emprunter plusieurs livres (avec une limite fixée, mais non précisée ici). Un livre peut être emprunté par plusieurs abonnés au fil du temps (mais jamais par plus d'un à la fois pour le moment). Un emprunt est caractérisé par une date d'emprunt et une date de retour prévue.

Le travail c'est d'identifier :

1. Les Entités Les Associations

2. Les Cardinalités

II- Niveau Conceptuel

Solution:

1. Entité : Abonné

- **Identifiant (Clé) : Numéro_Abonné** Nom
- Prénom
- Statut (Exemple : Étudiant, Professeur)
- Adresse

2. Entité : Livre

- **Identifiant (Clé) : ISBN**
- Titre
- Année_Publication
- Nombre_Pages

3. Entité : Auteur

- **Identifiant (Clé) : Code_Auteur**
- Nom_Auteur
- Prénom_Auteur

4. Entité : Emprunt

- **Identifiant (Clé) : ID_Emprunt**
- Date_Emprunt
- Date_Retour_Prévue
- Date_Retour_Réelle

II- Niveau Conceptuel

Solution:

1. Relation entre Livre et Auteur

Cette association s'appelle généralement **Écrire** ou **Rédiger**.

- Un **Livre** → **doit avoir au moins un** Auteur, et **peut en avoir plusieurs** (1, N).
- Un **Auteur** → **doit avoir écrit au moins un** Livre, et **peut en écrire plusieurs** (1, N).

2. Relation entre Abonné et Livre (pour l'Emprunt Actuel)

Cette association s'appelle généralement **Emprunter** ou **Réaliser**. Nous décrivons ici la situation à **un instant T** (qui a le livre maintenant).

- Un **Abonné** → **peut n'emprunter aucun** Livre, ou **en emprunter plusieurs** (0, N).
- Un **Livre** → **n'est emprunté par aucun** Abonné (s'il est sur l'étagère), ou **par un seul maximum** (car un seul exemplaire à la fois) (0, 1).

II- Niveau Conceptuel

Solution:

3. Relation entre Emprunt, Abonné et Livre

L'entité **Emprunt** est une entité qui **détaille** la relation d'emprunt. Elle est liée à la fois à l'Abonné et au Livre impliqués.

a) Emprunt et Abonné

- Un **Emprunt** → **doit concerner exactement un** Abonné (1, 1).
- Un **Abonné** → **peut n'avoir fait aucun** Emprunt (dans l'historique), ou **en avoir fait plusieurs** (0, N).

b) Emprunt et Livre

- Un **Emprunt** → **doit concerner exactement un** Livre (1, 1).
- Un **Livre** → **peut n'avoir fait l'objet d'aucun** Emprunt (dans l'historique), ou **de plusieurs** (0, N).

II- Niveau Conceptuel

2. Extensions au MCD :

Ces extensions permettent de capturer des contraintes et des structures hiérarchiques qui vont au-delà du formalisme Entité-Association de base.

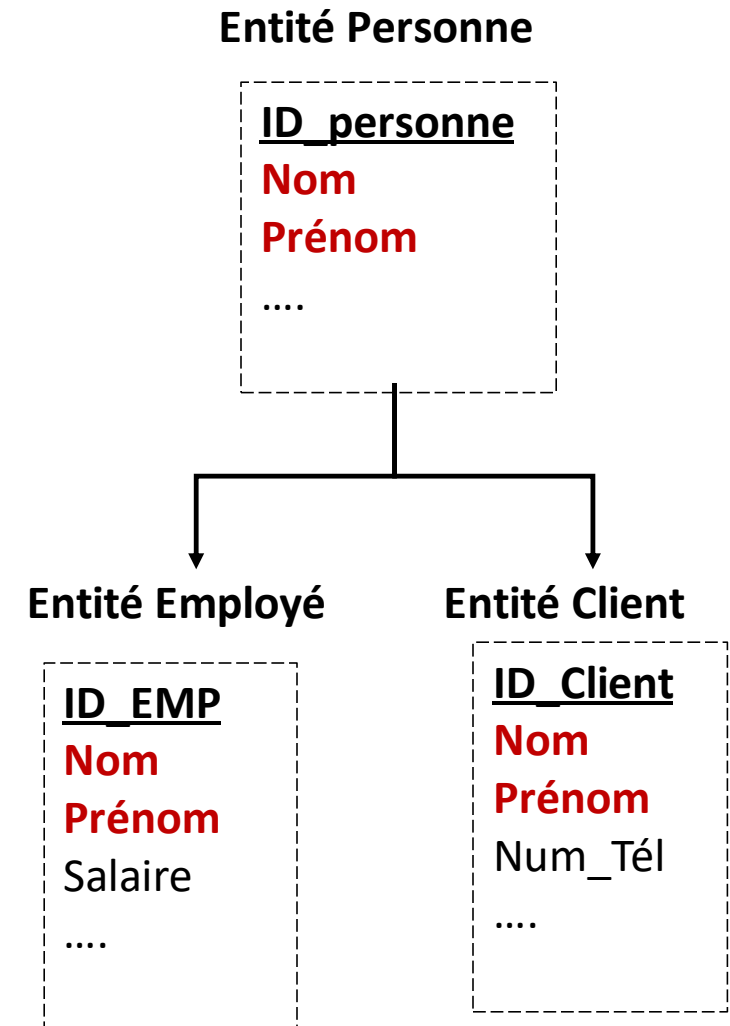
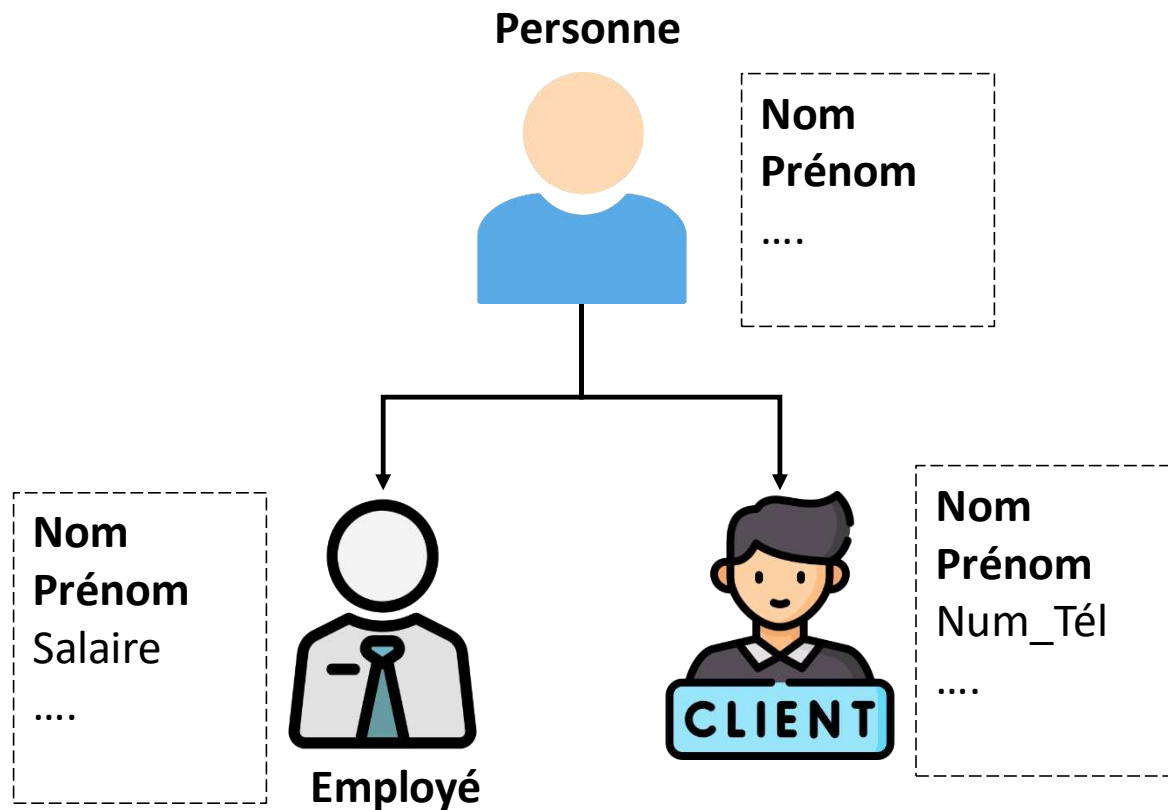
2.1 L'Héritage (Spécialisation/Généralisation): L'héritage est utilisé pour modéliser une relation de type "**est un**" entre différents types d'entités.

- **Généralisation** : On identifie des caractéristiques communes à plusieurs entités et on les regroupe dans une **Entité Mère (Générique)**.
- **Spécialisation** : L'Entité Mère est divisée en **Entités Filles (Spécifiques)** qui héritent de tous les attributs et associations de la mère, mais ajoutent leurs propres attributs et contraintes.

Entité Mère (Exemple)	Entités Filles (Héritiers)	Intérêt
PERSONNE (idPersonne, Nom, Adresse)	EMPLOYÉ (Salaire, DateEmbauche)	On évite de dupliquer Nom et Adresse pour Employé et Client.
	CLIENT (TauxRemise, DatePremiereCommande)	

II- Niveau Conceptuel

Héritage en réalité vs BD



II- Niveau Conceptuel

2.2 Contraintes Intra et Inter-Associations

Ces contraintes sont utilisées pour définir des règles de participation et de dépendance lorsque les entités sont impliquées dans **plusieurs associations simultanément**

2.2.1 Contraintes Inter-Associations (La Dépendance)

Cette contrainte exprime une **dépendance logique conditionnelle** entre deux associations **distinctes**. Elle relie la validité d'une participation à la validité d'une autre.

La participation d'une entité à l'**Association B** est subordonnée à sa participation à l'**Association A**.

- **Représentation** : Elles sont souvent notées textuellement à côté du modèle pour plus de clarté.
- **Règle** : Pour participer à B, il faut d'abord (ou simultanément) participer à A.

Exemple Simple de Dépendance

Considérons les entités **EMPLOYÉ**, **PROJET** et **LABORATOIRE**.

- Association 1 : **AFFECTER** (entre Employé et Projet)
- Association 2 : **ACCÉDER** (entre Employé et Laboratoire)

« Un Employé ne peut **Accéder** à un Laboratoire que s'il est **Affecté** à un Projet qui utilise ce Laboratoire »

II- Niveau Conceptuel

2.2.2 Contraintes Intra-Associations (Le Choix)

Ces contraintes s'appliquent à un ensemble de liens qui partent **de la même entité** vers des associations différentes. Elles définissent les règles de **participation au sein de ce groupe**.

A. La Contrainte d'Exclusivité ($\oplus$)

L'occurrence de l'entité ne peut participer qu'à **UNE SEULE** des associations du groupe. C'est un choix mutuellement exclusif.

- Règle : **A OU B**, mais jamais **A ET B**.
- Exemple : Un **Produit** est soit **VENDU** à un Client, soit **JETÉ** (à la poubelle), mais il ne peut être les deux.
 - Si le **Produit** est dans la liaison **VENDRE**, il est **exclu** de **JETER**.

La Contrainte de Totalité (T)

L'occurrence de l'entité **DOIT** participer à **AU MOINS UNE** des associations du groupe.

- Règle : **A OU B (ou les deux)**.
- Exemple : Un **Employé** doit être soit **AFFECTÉ** à un Projet **OU FORMÉ** dans un centre, mais il ne peut pas exister sans être dans l'une des deux.
 - **Note Clé** : Contrairement à l'exclusivité, l'Employé peut tout à fait être **AFFECTÉ ET FORMÉ** simultanément si aucune autre règle ne l'interdit. La Totalité impose seulement le minimum (au moins un rôle).

II- Niveau Conceptuel

2.3. Modèle Conceptuel des Traitements Analytiques (MCTA)

Le **Modèle Conceptuel des Traitements Analytiques (MCTA)** est l'outil du niveau conceptuel qui modélise la **dynamique** et l'**évolution** du système d'information. Contrairement au MCD qui répond au "QUOI" (les données statiques), le MCTA répond au "**QUAND**" et "**POURQUOI**" les données doivent changer.

Son rôle est de décrire les **règles d'évolution** des données et des processus métier de manière **purement fonctionnelle**, sans se préoccuper des contraintes d'organisation ou de la technologie utilisée.

Le MCTA décrit comment le système réagit aux événements. Il s'articule autour de trois composants principaux qui forment le cycle de traitement :

- **Événement (Le Déclencheur)**

L'**Événement** est un fait qui survient dans le système ou son environnement et qui nécessite une réaction. C'est l'élément **déclencheur**. Les événements peuvent être externes (ex: *Arrivée d'une commande*), internes (ex: *Fin de la saisie*), ou temporels (ex: *Début du mois*).

II- Niveau Conceptuel

- **Opération (L'Action)**

L'**Opération** est l'ensemble d'actions qui doit être exécuté en réponse à l'événement. Elle est **indivisible** et modifie l'état du système d'information. C'est le "**quoi faire**" : créer, modifier, ou supprimer des entités et des associations définies dans le MCD.

- **Règle d'Émission (La Condition)**

La **Règle d'Émission** est une condition logique qui doit être vérifiée, **en plus** de l'occurrence de l'événement, pour que l'Opération puisse se déclencher. Cette condition s'appuie généralement sur la valeur des attributs dans le MCD.

2.3.1 La Logique du Déclenchement

Une Opération ne se déclenche que si deux critères sont simultanément satisfaits :

1. **L'Événement survient.**
2. **La Règle d'Émission est VRAIE.**

Si ces deux conditions sont remplies (logique **ET**), l'Opération est exécutée. Le résultat de l'Opération peut lui-même générer un nouvel **Événement** qui déclenche la suite du processus.

II- Niveau Conceptuel

Une bibliothèque veut informatiser la gestion des livres, des adhérents et des emprunts. Chaque adhérent peut emprunter plusieurs livres, mais un livre ne peut être emprunté que par un seul adhérent à la fois. Pour chaque emprunt, on doit enregistrer la date d'emprunt et la date de retour prévue. Chaque livre possède un titre, un auteur et un code unique. Chaque adhérent a un numéro, un nom et un email.

T1 : Gérer les Adhérents

Événement déclencheur : un nouvel adhérent s'inscrit / un adhérent se met à jour / un adhérent est supprimé.

Actions :

- Enregistrer un nouvel adhérent (numéro, nom, email).
- Modifier les informations d'un adhérent.
- Supprimer un adhérent.

Résultat :

- Adhérent enregistré / mis à jour / supprimé.

II- Niveau Conceptuel

T2 : Gérer les Livres

Événement déclencheur : ajout, modification ou retrait d'un livre.

Actions :

- Enregistrer un nouveau livre (code, titre, auteur).
- Modifier les informations d'un livre.
- Supprimer un livre.

Résultat :

- Livre enregistré / mis à jour / supprimé.

T3 : Gérer les Emprunts

Événement déclencheur : un adhérent souhaite emprunter ou restituer un livre.

Actions :

- Vérifier la disponibilité du livre.
- Créer un emprunt avec la date d'emprunt et la date de retour prévue.
- Mettre à jour l'état du livre (emprunté / disponible).
- Enregistrer la restitution du livre

Résultat :

- Emprunt créé / mis à jour (retourné).
- Livre marqué comme disponible ou non.

II- Niveau Conceptuel

2.4. Cycle de Vie des Objets (CVO)

Le **Cycle de Vie des Objets (CVO)** est une technique de modélisation dynamique qui se concentre sur l'évolution d'une **seule entité clé** (appelée l'Objet) de sa création à sa destruction. Il est souvent représenté graphiquement par un diagramme d'états et de transitions.

Son rôle est de s'assurer que les opérations définies dans le MCTA ne sont exécutées que lorsque l'objet est dans un **état valide**.

2.4.1 Concepts Clés du CVOA

A. L'Objet et ses États

L'Objet est une occurrence de l'entité centrale dont on étudie le comportement dynamique (ex : l'entité Commande, Dossier_Client ou Projet).

Un État représente une phase stable et significative dans l'existence de cet objet. L'état détermine l'ensemble des règles de gestion applicables à l'objet à ce moment-là.

Exemple pour l'entité COMMANDE : Saisie en Cours, Validée, En Préparation, Livrée.

II- Niveau Conceptuel

B. Événements et Transitions

La **Transition** est le passage d'un état à un autre. Elle est toujours provoquée par un **Événement** (défini dans le MCTA) et peut exécuter une Opération associée.

La transition se modélise par la formule :

$$\text{État initial} \xrightarrow[\text{Opération}]{\text{Événement}} \text{État final}$$

C. Itération (Boucle sur l'État)

L'**Itération** se produit lorsqu'un événement est reçu et qu'une opération est effectuée, mais que cela **ne modifie pas l'état** de l'objet.

- **Règle** : L'événement provoque une boucle sur l'état lui-même.
- *Exemple* : La commande est dans l'état **Saisie en Cours**. L'événement *Ajout d'un article* modifie la commande (Opération : *Mise à jour du contenu*), mais elle reste dans l'état **Saisie en Cours** car elle n'est pas encore finalisée.

II- Niveau Conceptuel

D. Transition Conditionnelle

Une transition est dite **conditionnelle** lorsqu'un événement mène à **plusieurs états finaux possibles**, le choix étant déterminé par une **condition (Règle d'Émission)**.

- **Règle** : Le système teste une condition logique pour diriger l'objet vers le bon état.
- *Exemple* : L'événement *Vérification du Stock* survient lorsque la commande est **Validée**.
 - **SI** (Stock suffisant) → Transition vers l'état **En Préparation**.
 - **SINON** (Stock insuffisant) → Transition vers l'état **En Attente de Réapprovisionnement**

Exemple de CVO : L'Entité Commande

Le CVO modélise les phases par lesquelles passe une commande, de sa création à sa clôture.

1. Les États Significatifs

Nous définissons quatre états stables pour l'objet **COMMANDE** :

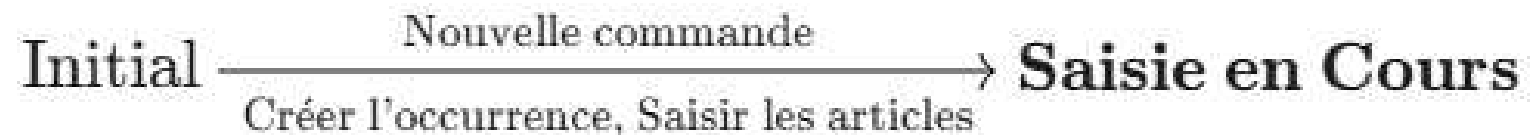
II- Niveau Conceptuel

État	Description
Saisie en Cours	La commande est en cours de création, elle est modifiable.
Validée	Le client a confirmé la commande (elle est figée, le paiement est lancé).
En Préparation	Le paiement est confirmé et les articles sont en cours d'emballage.
Livrée	La commande est reçue par le client (fin du cycle).

2. Les Transitions (Événements et Opérations)

Les transitions décrivent le passage d'un état à l'autre, déclenché par un événement.

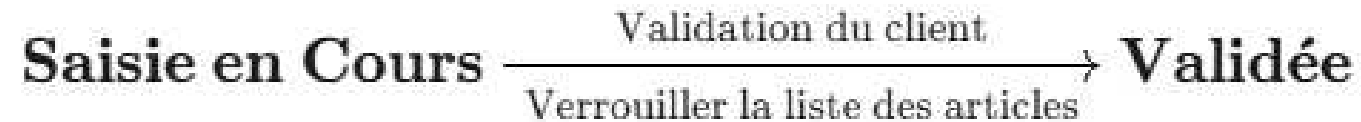
A. Création et Saisie



II- Niveau Conceptuel

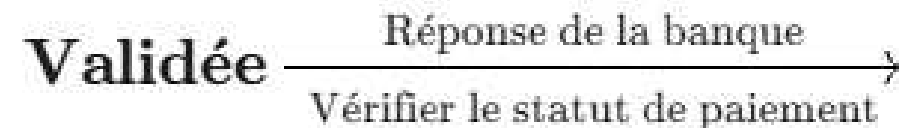
B. Validation

Le client finalise sa commande et lance le paiement.



C. Transition Conditionnelle (Vérification de Paiement)

Le système attend la réponse de la banque pour décider de la suite.

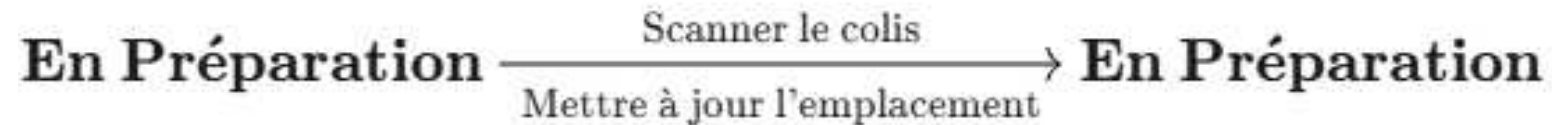


- SI (Paiement OK) : → En Préparation
- SINON (Paiement Échec) : → Annulée

II- Niveau Conceptuel

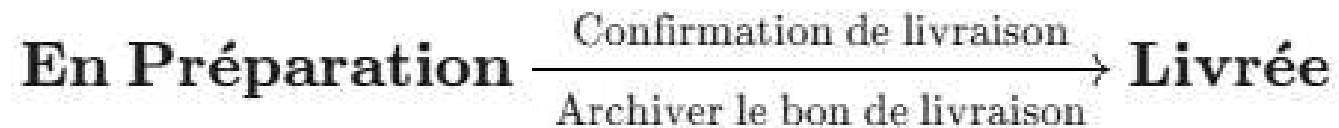
D. Itération (Boucle)

L'objet reste dans le même état malgré un événement.



E. Fin du Cycle

Le produit est livré.



Exercice:

Le service RH d'une grande entreprise gère deux catégories de personnel : les Employés et les Consultants Externes. Toutes les personnes (Employés et Consultants) sont enregistrées comme des Agents dans le système. Elles partagent des informations communes : un numéro d'identification unique, le nom, le prénom et la date d'entrée dans l'organisation (même pour les consultants). Les Employés ont des attributs spécifiques : le salaire annuel et le poste qu'ils occupent. Les Consultants ont des attributs spécifiques : le tarif journalier et le nom de leur société. L'entreprise gère des Projets. Un Agent (qu'il soit Employé ou Consultant) peut être affecté à plusieurs projets. Un Projet doit toujours avoir au moins un Agent affecté, et peut en avoir plusieurs.

- 1. Etablir le Modèle Conceptuel de Donnée (MCD)**
- 2. Etablir le Modèle Conceptuel des Traitements Analytiques (MCTA)**
- 3. Élaborer le cycle de vie d'un objet (CVO) correspondant à une entité de votre choix**

La solution:

1. MCD

Entités et associations

1. AGENT(entité mère commune aux employés et consultants)

Attributs :

- Num_Agent (Identifiant)
- Nom
- Prénom
- Date_Entrée

2. EMPLOYÉ(spécialisation de AGENT)

Attributs spécifiques :

- Salaire_Annuel
- Poste

3. CONSULTANT(spécialisation de AGENT)

Attributs spécifiques :

- Tarif_Journalier
- Société

4. PROJET

Attributs :

- Num_Projet (Identifiant)
- Nom_Projet
- Date_Début
- Date_Fin

La solution;

5. **AFFECTATION**(association entre AGENT et PROJET)

Cardinalités :

- Un Agent peut être affecté à 0, n projets
- Un Projet doit avoir au moins 1, n agents

Attributs possibles :

- Date_Affectation
- Rôle (optionnel)

2. MCTA

Événements (ce qui déclenche une opération)

- **E1** : Recrutement d'un nouvel agent
- **E2** : Enregistrement d'un consultant externe
- **E3** : Création d'un projet
- **E4** : Affectation d'un agent à un projet

La solution;

Opérations (traitements effectués)

- **O1** : Enregistrer un agent
- **O2** : Enregistrer les informations spécifiques (employé ou consultant)
- **O3** : Créer un projet
- **O4** : Affecter un agent à un projet

Résultats (produits ou états obtenus)

- **R1** : Agent enregistré
- **R2** : Employé ou consultant créé
- **R3** : Projet ajouté au système
- **R4** : Affectation validée

Objets manipulés

- **Agent**
- **Employé**
- **Consultant**
- **Projet**
- **Affectation**

La solution;

[E1 : Recrutement Agent]



(O1 : Enregistrer Agent) → [R1 : Agent enregistré]



|—> (O2 a: Enregistrer Employé) → [R2 : Employé créé]

|—> (O2 b: Enregistrer Consultant) → [R2 : Consultant créé]

[E3 : Création Projet]



(O3 : Créer Projet) → [R3 : Projet ajouté]

[E4 : Affectation Agent-Projet]



(O4 : Affecter Agent à Projet) → [R4 : Affectation validée]

3. CVO

Les meilleurs candidats pour un cycle d'objet ici sont les objets **Agent** et **Projet**, car **ils évoluent** dans le système.

Cycle d'objet : AGENT

[Agent créé]

↓ (O1 : Enregistrer agent)

[Agent actif]

↓ (O4 : Affecter à un projet)

[Agent affecté à projet]

↓ (O5 : Fin de mission ou départ)

[Agent inactif]

Cycle d'objet : PROJET

[Projet créé]

↓ (O3 : Créer projet)

[Projet en cours]

↓ (O4 : Affectation d'agents)

[Projet actif]

↓ (O6 : Clôture projet)

[Projet terminé]

III- Niveau organisationnel

1. Introduction au Modèle Organisationnel des Traitements Analytiques (MOTA)

Définition et rôle: Le **Modèle Organisationnel des Traitements Analytiques (MOTA)** représente le **niveau organisationnel** de la conception d'un système d'information. Après avoir modélisé **les données** (à l'aide du MCD) et **les traitements conceptuels** (avec le MCTA), le MOTA permet de préciser **comment ces traitements sont organisés dans l'entreprise, qui les exécute, où et dans quel ordre.**

Le MOTA est un modèle qui décrit **l'organisation des traitements** du système d'information. Il met en évidence :

- Les **acteurs** (humains, services, ou systèmes automatiques) qui réalisent les traitements ;
- Les **interactions** entre ces acteurs ;
- Les **supports d'information** utilisés (documents, bases de données, formulaires, etc.) ;
- La **circulation des informations** entre les différentes unités organisationnelles.

Son rôle principal est de **passer d'une vision fonctionnelle à une vision organisationnelle** du système, afin de préparer la mise en œuvre concrète des processus identifiés au niveau conceptuel.

En résumé, le **MOTA** répond à la question :
« **Qui fait quoi, où et quand, à partir de ce que le MCTA a défini comme traitements à effectuer ?** »

Le MOTA s'inscrit dans la **continuité du MCTA**, mais sur un plan plus concret :

- Le **MCTA** décrit **ce qu'il faut faire** (les traitements et actions sur les données).
- Le **MOTA** décrit **comment ces traitements sont exécutés** dans l'organisation (répartition entre acteurs, services, ou systèmes).

Le **Cycle de Vie des Objets (CVO)** complète le MOTA en représentant **l'évolution des objets** au cours des différentes actions définies dans les traitements.

- Le **MOTA** montre **qui réalise les actions et comment elles s'enchaînent**.
- Le **CVO** montre **comment l'objet évolue à travers ces actions et états**.

Ces deux modèles sont donc **complémentaires** : le MOTA se concentre sur **l'organisation des processus**, tandis que le CVO décrit **la dynamique des objets manipulés**.

Exemple simple

Prenons l'exemple d'un **système de gestion de commandes en ligne** :

- Le **MCTA** indique qu'il faut *créer, valider et expédier* une commande.
- Le **MOTA** précise que :
 - Le **client** crée la commande,
 - Le **service commercial** la valide,
 - Le **service logistique** la prépare et l'expédie.

Le MOTA permet donc de visualiser **l'organisation réelle** de ces traitements au sein de l'entreprise.

2. Les éléments constitutifs du MOTA

Le **Modèle Organisationnel des Traitements Analytiques (MOTA)** s'appuie sur plusieurs éléments fondamentaux qui permettent de représenter de manière précise l'organisation des traitements dans le système d'information. Ces éléments assurent la cohérence entre les activités, les acteurs et les informations manipulées.

- **Les acteurs**

Les **acteurs** représentent les entités (humaines ou automatiques) qui interviennent dans la réalisation des traitements. Ils peuvent être :

- Des **personnes** (utilisateurs, employés, responsables de service) ;
- Des **unités organisationnelles** (service commercial, service comptable, logistique, etc.) ;
- Des **systèmes informatiques** (applications, serveurs, automates).

Chaque acteur possède un **rôle** précis et des **responsabilités** dans l'exécution des tâches.

- **Les traitements**

Les **traitements** sont les **tâches, opérations ou actions** réalisées par les acteurs sur les données du système. Ils traduisent les activités identifiées au niveau conceptuel (MCTA) dans un contexte organisationnel.

Exemples :

- Créer une commande,
- Valider un paiement,
- Mettre à jour le stock,
- Envoyer une facture.

Chaque traitement peut être :

- **Manuel** (réalisé par un acteur humain),
- **Automatique** (exécuté par un programme ou un système),
- **Mixte** (partagé entre un utilisateur et un outil informatique).

- **Les supports d'information**

Les **supports d'information** regroupent tous les **documents, fichiers, formulaires, ou bases de données** nécessaires à l'exécution des traitements. Ils peuvent être :

- **Physiques** : formulaires papier, bons de commande, factures, etc.
- **Numériques** : fichiers informatiques, tables de bases de données, documents électroniques.

Ces supports assurent la **traçabilité** et la **circulation des informations** entre les différents acteurs.

- **Les flux d'information**

Les **flux d'information** représentent les **échanges de données** entre les acteurs et les traitements. Ils indiquent :

- Qui envoie l'information,
- Qui la reçoit,
- Sous quelle forme elle circule.

Les flux permettent de visualiser la **communication entre les différents services** et d'identifier les **points d'interconnexion** entre traitements.

- **Les règles d'organisation**

Les **règles d'organisation** définissent les **contraintes** qui régissent le déroulement des traitements.

Elles peuvent être :

- **Temporelles** : ordre d'exécution, délais, synchronisation entre tâches.
- **Hiérarchiques** : validation par un supérieur, enchaînement entre services.
- **Fonctionnelles** : conditions de déclenchement, priorités ou dépendances.

Ces règles assurent la **cohérence du fonctionnement global** du système et garantissent la conformité avec les procédures internes de l'entreprise.

4. Complémentarité entre le MCTA et le MOTA

Le **MOTA** et le **MCTA** interviennent dans la **phase de modélisation des traitements analytiques**, après la construction du modèle de données (MCD/MLD).

MCD → MLD → MCTA → MOTA

- **MCD (Modèle Conceptuel de Données)** : décrit les entités et leurs relations (ex. Client, Produit, Vente).
- **MLD (Modèle Logique de Données)** : traduit le MCD sous forme de tables relationnelles dans la base de données.
- **MCTA (Modèle Conceptuel des Traitements Analytiques)** : définit quels calculs ou analyses seront faits sur ces données.
- **MOTA (Modèle Organisationnel des Traitements Analytiques)** : définit qui fait quoi, avec quel outil, et quand ces traitements sont exécutés.

Donc on dit que Le MOTA est une continuité du MCTA :

- Le MCTA définit **le contenu analytique** (quoi faire)
- Le MOTA définit **l'organisation du traitement** (comment le faire)

Exemple: une entreprise de vente

On considère une entreprise de vente composée de trois entités principales : **Client**, **Commande** et **Produit**.

- **Client** (*id_client, nom, adresse*)
- **Commande** (*id_commande, date_commande, id_client*)
- **Produit** (*id_produit, nom_produit, prix*)

Relations :

- Un **client** peut passer plusieurs **commandes**.
- Une **commande** peut contenir plusieurs **produits**, et un **produit** peut apparaître dans plusieurs **commandes**.

- Client (1,n) — passe — (1,n) Commande
- Commande (1,n)— contient — (0,n) Produit

MCTA:

- **Événement : Création d'un client**

- Nouveau client enregistré dans le système
- Opérations : enregistrement des informations et vérification de l'unicité de l'email
- Résultat attendu : client ajouté à la table **Client**

- **Événement : Création d'une commande**

- Un client passe une nouvelle commande
- Opérations : saisie des informations, association du client et des produits
- Résultat attendu : commande ajoutée à la table **Commande**

- **Événement : Ajout d'un produit à une commande**

- Sélection d'un ou plusieurs produits pour une commande existante
- Opérations : calcul du montant total (prix × quantité)
- Résultat attendu : commande complète prête pour validation

- **Événement : Validation d'une commande**

- La commande est confirmée par le client
- Opérations : mise à jour du statut de la commande, déclenchement du calcul du chiffre d'affaires
- Résultat attendu : commande validée et comptabilisée

- **Événement : Analyse des ventes**

- Action analytique périodique (ex : fin de mois)
- Opérations : agrégation des ventes par produit et par mois, calcul du chiffre d'affaires total
- Résultat attendu : rapport analytique mensuel, indicateurs de performance

MOTA:

Le **MOTA** décrit **comment les événements et opérations du MCTA sont organisés et exécutés** pour produire les analyses décisionnelles.

Étape 1 : Sources de données

- Les données viennent des **tables de gestion** du MCD :
 - **Client** (id_client, nom, adresse)
 - **Commande** (id_commande, date_commande, id_client)
 - **Produit** (id_produit, nom_produit, prix)
 - **Ligne_Commande** (id_commande, id_produit, quantité)
- Ces tables sont les **entrées pour tous les traitements analytiques**.

Étape 2 : Traitements analytiques

Pour chaque événement du MCTA, le MOTA décrit **les étapes de calcul et organisation des données** :

1. Création d'un client

1. Vérification et enregistrement des informations dans la table **Client**
2. Préparation pour pouvoir associer les commandes futures

2. Création d'une commande

1. Vérification que le client existe
2. Saisie des informations de commande
3. Association avec le ou les produits sélectionnés

3. Ajout d'un produit à une commande

1. Calcul du montant total (prix × quantité)
2. Enregistrement dans **Ligne_Commande**

4. Validation d'une commande

1. Mise à jour du statut de la commande
2. Déclenchement des **agrégations analytiques** (calcul du chiffre d'affaires, suivi des indicateurs)

5. Analyse des ventes

1. Agrégation des ventes par produit et par mois
2. Calcul du chiffre d'affaires total par période
3. Préparation des rapports pour le tableau de bord

Étape 3 : Résultats analytiques

- **Tableau de bord des ventes mensuelles** par produit
- **Nombre de commandes par client**
- **Classement des produits les plus vendus**
- **Chiffre d'affaires total** de la boutique

Le flux logique:

[Tables MCD : Client, Commande, Produit, Ligne_Commande]



[Création client / commande / ajout produit]



[Validation commande → déclenchement calculs]



[Agrégations et calculs analytiques]



[Résultats / Tableaux de bord / Rapports]

- Le MOTA montre comment les événements et opérations du MCTA sont organisés pour produire les indicateurs.
- Il fait le lien concret entre les données du MCD et les analyses décisionnelles.
- On peut ensuite passer à l'implémentation (SQL, Power BI, Excel, etc.) à partir du MOTA

6. Différence simple entre MCTA et MOTA

Aspect	MCTA	MOTA
Objectif	Décrire les événements analytiques	Décrire comment organiser les traitements analytiques pour obtenir les résultats
Question à se poser	Qu'est-ce que je veux analyser ?	Comment je vais produire cette analyse ?
Contenu	Événements métiers, actions, résultats	Sources de données, étapes de calcul, flux, périodicité, résultats
Limitation principale	Ne montre pas comment exécuter les calculs ni comment organiser les données	Permet de planifier et organiser concrètement les traitements

Petit exemple concret avec les tables

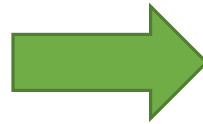
Tables de base (MCD):

- **Client** (id_client, nom, adresse)
- **Commande** (id_commande, date_commande, id_client)
- **Produit** (id_produit, nom_produit, prix)
- **Ligne_Commande** (id_commande, id_produit, quantité)

a) MCTA : “quoi analyser ?

Événement : Création d'une commande

Événement : Création d'un client



MCTA te dit seulement “quoi analyser”, pas comment ni dans quel ordre faire les calculs

Limitation: On sait **ce qu'on veut**, mais on **ne sait pas encore comment l'obtenir** :

- Par quelles tables exactement ?
- Dans quel ordre faire les calculs ?
- Comment organiser les agrégations ?

b) MOTA : “comment organiser ?”

On prend les événements du MCTA et on décrit **le traitement concret** :

- Sources : tables Client, Commande, Produit, Ligne_Commande

Étapes :

1. Valider commande
2. Calculer montant total (prix × quantité)
3. Agréger par client et par mois
4. Générer le rapport ou le tableau de bord



MOTA résout les limites du MCTA : il organise les calculs, précise les flux de données et montre comment produire les indicateurs.

Exercice :

Une bibliothèque souhaite analyser son système d'information pour mieux gérer ses prêts et ses adhérents.

Les tables suivantes sont utilisées dans la base de données :

1. Adhérent (*id_adherent, nom, ville*)

2. Livre (*id_livre, titre, auteur*)

3. Emprunt (*id_emprunt, id_adherent, id_livre, date_emprunt, date_retour*)

Partie 1 : MCD

- Dessinez le **MCD simplifié** avec ces 3 tables.
- Indiquez les **relations** :
 - Un adhérent peut emprunter plusieurs livres
 - Un livre peut être emprunté par plusieurs adhérents (mais pas en même temps)

Partie 2 : MCTA

- Identifiez **3 événements analytiques** possibles dans cette bibliothèque.
- Pour chaque événement, indiquez :
 - L'**opération**
 - La **résultat**

Partie 3 : MOTA

- Pour un des événements du MCTA, décrivez **comment le traitement sera organisé** :
 - Quelles **tables** seront utilisées ?
 - Quelles **opérations** seront effectuées ?
 - Quel sera le **résultat final** ?

MCTA :

•Événement : Emprunt d'un livre

- **Opération** : calculer la durée du prêt, vérifier si le livre est en retard
- **Résultat** : le livre est rendu à temps ou en retard

•Événement : Retour d'un livre

- **Opération** : calculer le retard (nombre de jours dépassés)
- **Résultat** : retard = 3 jours, à facturer éventuellement

•Événement : Inscription d'un nouvel adhérent

- **Opération** : enregistrer les informations (nom, ville) dans la table Adhérent
- **Résultat** : adhérent ajouté et prêt à emprunter des livres

•Événement : Consultation des emprunts mensuels

- **Opération** : compter le nombre de prêts par adhérent ou par livre
- **Résultat** : "Adhérent A a emprunté 5 livres ce mois-ci"

MOTA :

Le MOTA décrit comment les événements du MCTA se déroulent concrètement et produisent les résultats.

1. Sources de données

Les tables du MCD :

- **Adhérent** (*id_adherent, nom, ville*)
- **Livre** (*id_livre, titre, auteur*)
- **Emprunt** (*id_emprunt, id_adherent, id_livre, date_emprunt, date_retour*)

2. Organisation des traitements

Événement 1 : Emprunt d'un livre

1. **Tables utilisées** : Emprunt, Adhérent, Livre
2. **Opérations / Calculs** :
 1. Vérifier que l'adhérent existe
 2. Enregistrer la date de l'emprunt et le livre dans la table **Emprunt**
3. **Résultat attendu** : livre associé à l'adhérent, prêt enregistré

Événement 2 : Retour d'un livre

1. **Tables utilisées** : Emprunt, Livre
2. **Opérations / Calculs** :
 - Calculer la durée du prêt ($\text{date_retour} - \text{date_emprunt}$)
 - Vérifier si le livre est rendu en retard
3. **Résultat attendu** : mise à jour de l'état du livre, identification des retards éventuels

Événement 3 : Inscription d'un nouvel adhérent

1. Tables utilisées : Adhérent

2. Opérations / Calculs :

- Enregistrer les informations de l'adhérent
- Vérifier la validité des données (ex : nom non vide)

3. Résultat attendu : adhérent ajouté et prêt à effectuer des emprunts

Événement 4 : Consultation des emprunts mensuels

1. Tables utilisées : Emprunt, Livre, Adhérent

2. Opérations / Calculs :

- Compter le nombre d'emprunts par adhérent ou par livre
- Agréger les résultats par mois ou par catégorie

3. Résultat attendu : rapport mensuel des emprunts pour la bibliothèque

7. Périodicité dans le MOTA

Définition : La fréquence à laquelle un traitement analytique est exécuté, permettant de planifier les traitements réguliers et d'organiser les rapports.

Cas possibles :

- **Temps réel / événementiel** : déclenché immédiatement par l'événement (ex : enregistrement d'un emprunt ou retour) → pas besoin de périodicité.
- **Quotidien** : traitements ou rapports réalisés chaque jour (ex : liste des emprunts du jour)
- **Hebdomadaire** : agrégations ou statistiques sur la semaine (ex : nombre total de prêts par adhérent)
- **Mensuel** : rapports synthétiques sur le mois (ex : tableau des retards mensuels)

IV- Niveau logique

1. Introduction au MLDR

1.1. Qu'est-ce qu'une base de données répartie ?

Une base de données répartie est un système où **les données ne sont pas stockées en un seul endroit**, mais sont **réparties sur plusieurs sites physiques** (serveurs, villes, pays...).

Chaque site peut :

- Stocker une partie des données,
- Avoir des copies (réplication),
- Gérer localement les requêtes.

Exemple : une entreprise avec des agences à Casablanca, Rabat et Paris

1.2. BD centralisée vs BD répartie

Critère	BD Centralisée	BD Répartie
Stockage	Un seul site	Plusieurs sites
Performances	Souvent limitées	Accès plus rapide localement
Tolérance aux pannes	Faible	Haute
Complexité	Simple	Complexe
Communications	Faible	Importante

1.3. Pourquoi répartir les données ?

- **Proximité des utilisateurs** → accès plus rapide
- **Performance** → traitement local
- **Disponibilité** → un site tombe, les autres fonctionnent
- **Sécurité** → répartir les risques
- **Réduction des coûts** → éviter gros serveurs centralisés
- **Tolérance aux pannes**

1.4. Où se situe le MLDR (Modèle Logique des Données Réparties) dans la démarche Merise ?

Cycle de conception Merise :

- 1.MCD – Modèle Conceptuel des Données
- 2.MLD – Modèle Logique des Données
- 3.MLDR – Modèle Logique des Données Réparties
- 4.MPD – Modèle Physique des Données (réparti ou non)

Le MLDR intervient après le MLD, quand on doit décider où vont être stockées les données

1.5. Définition du MLDR

Le **MLDR** = **modèle logique du système distribué**, qui décrit :

- Comment les données sont **fragmentées**,
- Comment elles sont **répliquées**,
- Comment elles sont **allouées aux sites**.

C'est le schéma **logique ET distribué** de la base.

2. MLDR (Modèle Logique des Données Réparties)

MLDR = *Modèle Logique des Données Réparties*.

C'est l'extension du **MLD** dans un contexte de **base de données distribuée**.

Il précise **comment les données logiques sont réparties physiquement entre plusieurs sites**

2.2. Rôle du MLDR

Le MLDR sert à définir :

- **Quels fragments de données** sont créés
- **Quel type de fragmentation** est utilisé (horizontale, verticale, mixte...)
- **Quels sites** stockent quels fragments
- **Les copies (réplication)**
- **Les règles de reconstruction** (comment retrouver la table complète)

2.3. Différence entre MLD et MLDR

MLD	MLDR
Structure logique des données	Structure logique distribuée
Décrit les tables, clés, types	Décrit fragmentation, réplication, allocation
Indépendant du lieu	Dépend de la distribution des sites
Ne gère pas la localisation	Gère la localisation des fragments

Le MLDR ne modifie pas la structure des données, mais modifie leur organisation dans l'espace

2.4. Objectifs du MLDR

Le MLDR vise à optimiser :

- **Performance** → données proches des utilisateurs
- **Disponibilité** → plusieurs sites peuvent répondre
- **Tolérance aux pannes** → réplication
- **Coût des communications** → éviter les requêtes inter-sites
- **Sécurité** → fragmentation stratégique
- **Répartition de charge**

2.5. Ce que contient un MLDR

Un MLDR complet décrit :

1. Les sites (ex : Casablanca, Rabat, Paris)

2. Les types de fragments

1. horizontaux
2. verticaux
3. mixtes

3. Les règles de fragmentation: prédicats, regroupements

4. La réplication: totale, partielle ou aucune

5. L'allocation: quel site contient quel fragment

6. Les règles de reconstruction: UNION, JOIN, clés communes

Exemple:

MLD :

Clients
<u>ID</u> Nom Ville

MLDR :

- **Site Casablanca** : CLIENTS_CASA = CLIENTS où Ville = 'Casablanca'
- **Site Rabat** : CLIENTS_RABAT = CLIENTS où Ville = 'Rabat'
- **Site Paris** : copie complète (réplication totale)

3. Concepts fondamentaux du MLDR

3.1. Les Sites (ou Nœuds)

Un *site* est un emplacement où la base ou une partie de la base est stockée.

Il peut s'agir :

- D'un serveur dans une ville (Casablanca)
- D'un datacenter (Paris)
- D'un serveur cloud (AWS EU-West)

Les sites peuvent être :

- **Autonomes** (chacun gère sa partie)
- **Coordonnés** (contrôlés par un gestionnaire global)

3.2. La Fragmentation des Données

La fragmentation consiste à **découper une table en plusieurs parties** appelées *fragments*, qui seront distribués sur les sites.

Cela permet :

- D'avoir des données proches des utilisateurs
- De réduire les communications inter-sites
- De paralléliser le traitement
- De limiter les accès à des données sensibles

Chaque fragment doit permettre la reconstruction de la table d'origine.

3.2.1. Fragmentation Horizontale

Découper une table en **lignes (tuples)** selon un critère.

Exemple :

CLIENTS_CASA = CLIENTS où Ville = 'Casablanca'

CLIENTS_RABAT = CLIENTS où Ville = 'Rabat'

- Chaque fragment contient les mêmes colonnes
- Union des fragments = table complète
- Reconstruction par UNION



Avantages :

- Accès local rapide
- Logique géographique facile

3.2.2. Fragmentation Verticale

Découper une table en **colonnes (attributs)**.

Exemple :

CLIENTS_IDENT(ID, Nom, Prénom)

CLIENTS_CONTACT(ID, Adresse, Tél)

- Fragmentation par attributs
- Chaque fragment conserve la clé primaire
- Reconstruction par JOIN



Avantages :

- Sécurité (séparer données sensibles)
- Optimisation des lectures partielles

3.2.3. Fragmentation Mixte

Combinaison horizontale + verticale.

Exemple :

- Fragmenter d'abord par région
- Puis chaque région en colonnes



Avantages :

- Très flexible
- Optimal pour grands systèmes

3.2.4. Fragmentation Dérivée

Fragmenter une table à partir d'une autre table.

Exemple :

- CLIENTS est fragmenté par ville
- COMMANDES suit le même schéma

3.4. Allocation des Fragments

Après fragmentation, on doit décider **sur quel site placer quel fragment**.

Critères d'allocation :

- **Proximité des utilisateurs** (data locality)
- **Fréquence d'accès**
- **Sécurité** (mettre les données sensibles dans un site sécurisé)
- **Coût de transfert**
- **Puissance des sites** (équilibrer la charge)

5. Représentation Graphique du MLDR

Il n'existe **pas de standard officiel** (comme UML), mais trois représentations sont largement utilisées :

5.1. Représentation par “Tables + Sites”

C'est la représentation **la plus simple et la plus pédagogique**.

On représente :

- Un rectangle pour chaque **site**,
- Des sous-tables pour chaque **fragment**,
- Avec les noms de fragments à l'intérieur.

Exemple :

Site CASA

Clients_CASA(ID, Nom, Tél)

Site RABAT

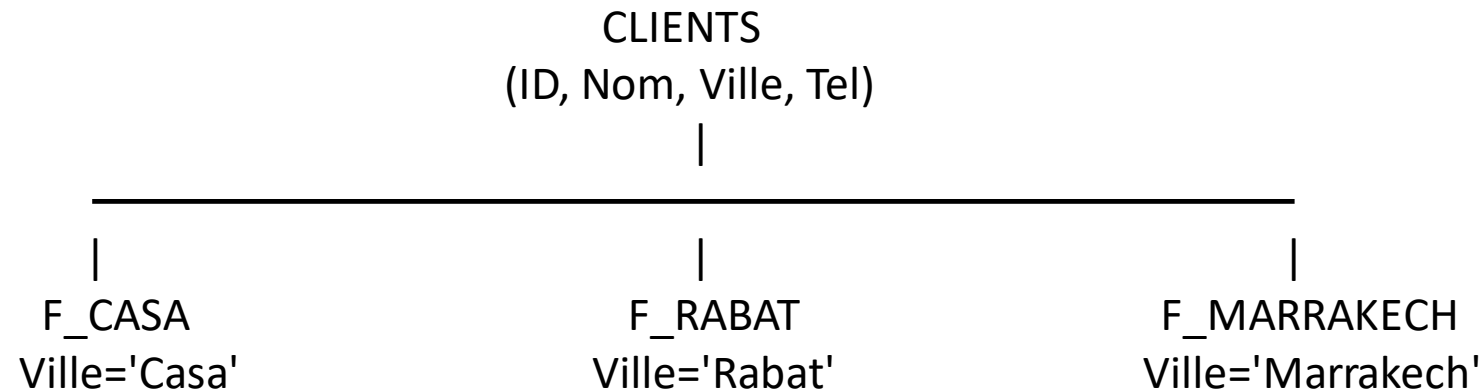
Clients_RABAT(ID, Nom, Tél)

Très clair pour montrer la **distribution et la réplication**.

5.2. Représentation par Schéma de Fragmentation

On représente la table source et les flèches vers les fragments.

Exemple fragmentation horizontale :



6. Les étapes de construction d'un MLDR

6.1 Analyse du contexte organisationnel

Identifier les **sites / nœuds** où les données seront stockées :

- Site géographique A, B, C...
- Data center interne, cloud, filiale, etc.



Déterminer :

- Les acteurs utilisant chaque site.
- Les applications exécutées sur chaque site.
- Les contraintes (bande passante, sécurité, confidentialité).

6.2 Identification des besoins de distribution

Pour chaque entité du MLD :

- Quels sites ont besoin de la consulter ?
- Quels sites ont besoin de la mettre à jour ?

Définir les impératifs :

- **Disponibilité locale** des données.
- **Performance** (éviter les requêtes inter-sites trop fréquentes).
- **Sécurité** (isolement des données sensibles).
- **Règlementation** (stockage local, RGPD, loi nationale...).

6.3 Choix du mode de fragmentation

Pour chaque table du MLD, choisir entre :

a) Fragmentation horizontale

- Découpage par lignes.
- Exemple : Clients par région → Clients_Nord / Clients_Sud.

b) Fragmentation verticale

- Découpage par colonnes.
- Exemple : Séparer données sensibles (num CIN) dans un site sécurisé.

c) Fragmentation mixte

- Combinaison des deux.

d) Pas de fragmentation

- Réplication ou centralisation complète.

6.4 Définition des fragments

Pour chaque entité :

- Nom du fragment.
- Type de fragmentation.
- Condition (sélection / projection).
- Attributs inclus.
- Contraintes à respecter :
 - Clés primaires.
 - Clés étrangères.
 - Dépendances fonctionnelles.

6.5 Mise en place de la réplication

Si une donnée doit exister sur plusieurs sites, choisir :

a) Réplication totale

- Copie complète sur chaque site.
- **Avantages** : disponibilité + performances.
- **Inconvénients** : synchronisation coûteuse.

b) Réplication partielle

- Certains fragments seulement sont répliqués.

c) Pas de réplication (distribution exclusive)

6.6 Affectation des fragments aux sites

- Associer chaque fragment à un ou plusieurs sites.
- Renseigner dans le MLDR :
 - Fragment F1 → Site A
 - Fragment F2 → Site B et C
 - Fragment F3 → Sites A, B (réplication)

Chaque fragment devient une *table logique* associée à un site.

6.7 Validation de la cohérence

Vérifier :

- La reconstruction des entités (operator **UNION, JOIN**).
- Le maintien des clés étrangères malgré la distribution.
- La minimisation des communications inter-sites.
- La compatibilité avec le SGBD réparti (Oracle, SQL Server, etc.)

6.8 Génération du schéma logique réparti

Compilation finale du MLDR :

- Liste des fragments.
- Leur structure.
- Leur localisation.
- Les règles de synchronisation et d'accès

Exemple simple:

On a trois tables dans le MLD centralisé :

ETUDIANT(idEtu, nom, prenom, ville)

MODULE(idMod, nomMod)

INSCRIPTION(idEtu*, idMod*, note)

Et trois **sites** répartis :

- **Site A : Casablanca**
- **Site B : Rabat**
- **Site C : Marrakech**

Étape 1 : Regarder la donnée = sur quel site elle doit aller ?

Dans le cahier de charge, on avait :

- Site A gère les étudiants de **Casablanca**
- Site B gère les étudiants de **Rabat**
- Site C gère les étudiants de **Marrakech**

Donc → la table **ETUDIANT** doit être **découpée (fragmentée)** selon la **ville**.

Fragmentation horizontale = on découpe par lignes (par lignes = par enregistrements).

ETUDIANT_A :

toutes les colonnes

condition : ville = 'Casablanca'

ETUDIANT_B :

toutes les colonnes

condition : ville = 'Rabat'

ETUDIANT_C :

toutes les colonnes

condition : ville = 'Marrakech'

Fragment	Condition	Site
ETUDIANT_A	ville = Casablanca	A
ETUDIANT_B	ville = Rabat	B
ETUDIANT_C	ville = Marrakech	C

Tous les sites ont besoin de MODULE.

Réplication totale

Cela veut dire : une copie complète sur A, B et C.

MODULE_A = copie complète de MODULE

MODULE_B = copie complète de MODULE

MODULE_C = copie complète de MODULE

Maintenant la table INSCRIPTION

Rappel : INSCRIPTION contient les notes des étudiants dans les modules.

Comme un étudiant est dans son site, ses INSCRIPTIONS doivent aller dans le **même site**.

Donc on fait une **fragmentation dérivée** depuis ETUDIANT.

INSCRIPTION_A :

contient les inscriptions des étudiants qui sont dans
ETUDIANT_A

INSCRIPTION_B :

contient les inscriptions des étudiants qui sont dans
ETUDIANT_B

INSCRIPTION_C :

contient les inscriptions des étudiants qui sont dans
ETUDIANT_C

Résumé simple

ETUDIANT → Fragmentation horizontale

Découpage par ville.

MODULE → Réplication

Copie sur chaque site.

INSCRIPTION → Fragmentation dérivée

On met chaque inscription dans le site de son étudiant.

Exemple 2 :

Une entreprise possède **3 agences** :

- **Site A : Casablanca**
- **Site B : Tanger**
- **Site C : Agadir**

Elle gère ses employés, ses départements et les affectations.

- Chaque site gère ses propres employés.
- Les départements doivent être disponibles sur tous les sites.
- Les affectations doivent être stockées dans le même site que l'employé.

Fragmentation de EMPLOYE

On utilise le critère **ville**.

C'est de la **fragmentation horizontale**.

EMPLOYEE_A (Casablanca)

EMPLOYEE_A(idEmp, nom, prenom, ville)
Condition : ville = 'Casablanca'

EMPLOYEE_B (Tanger)

EMPLOYEE_B(idEmp, nom, prenom, ville)
Condition : ville = 'Tanger'

EMPLOYEE_C (Agadir)

EMPLOYEE_C(idEmp, nom, prenom, ville)
Condition : ville = 'Agadir'

Fragmentation de DEPARTEMENT

Tous les sites doivent lire les départements → On fait une **réplication totale**.

DEPARTEMENT_A(idDep, nomDep)

DEPARTEMENT_B(idDep, nomDep)

DEPARTEMENT_C(idDep, nomDep)

Fragmentation de AFFECTATION

Une affectation correspond à **un employé dans un département**. Donc, son fragment doit être **dans le site où se trouve l'employé**. C'est une **fragmentation dérivée** à partir de EMPLOYE.

AFFECTATION_A: AFFECTATION_A(idEmp, idDep, dateDebut)

Condition : idEmp ∈ EMPLOYE_A

AFFECTATION_B: AFFECTATION_B(idEmp, idDep, dateDebut)

Condition : idEmp ∈ EMPLOYE_B

AFFECTATION_C: AFFECTATION_C(idEmp, idDep, dateDebut)

Condition : idEmp ∈ EMPLOYE_C

Modèle Logique des Traitements Répartis (MLTR)

Définition: le **Modèle Logique des Traitements Répartis (MLTR)** est un **modèle conceptuel** utilisé en conception de systèmes d'information pour **décrire et organiser les traitements** (processus, règles, flux, opérations) de manière **logique et indépendante** de toute technologie.

Il répond à la question :

Quels traitements doivent être réalisés, par qui, sur quelles données, et dans quel ordre ?

2. Objectifs du MLTR

Le MLTR sert à :

- Décomposer les traitements du système en **processus logiques**.
- Définir les **échanges d'informations** (messages, flux).
- Identifier les **acteurs** qui réalisent les traitements.
- Préparer la répartition future des traitements dans le système (architecture logicielle).
- Garantir une cohérence entre les traitements et les données (MCD/MLD).

Il constitue un **pont** entre l'analyse métier et la conception technique.

3. Concepts de base du MLTR

3.1 Processus

- Un ensemble d'opérations logiques cohérentes.
- Exemples : *Gérer une commande, Authentifier un utilisateur.*

3.2 Opérations

Une opération est :

- Atomique (indivisible)
- Déclenchée par un événement
- Réalisant un résultat métier

Exemples :

- *Créer commande*
- *Valider paiement*
- *Mettre à jour stock*

3.3 Acteurs

Rôles internes ou externes qui déclenchent ou exécutent des opérations.

Exemples :

- Client
- Administrateur
- Système de paiement

3.4 Flux / Échanges

Informations circulant entre les processus ou entre un acteur et un processus.

Exemple : *Demande de commande, Résultat validation, Accusé de réception.*

3.5 Règles de gestion

Éléments métier déterminant les conditions ou modes de fonctionnement.

Exemples :

- Une commande est validée si le stock est suffisant.
- Un client doit être authentifié avant un achat.

4. Représentation du MLTR

Généralement sous forme :

- **De schéma** (boîtes + flèches)
- **Ou de tableau structuré.**

Processus	Opération	Acteur	Flux entrant	Flux sortant	Règles de gestion
-----------	-----------	--------	--------------	--------------	-------------------

5. Méthodologie de construction du MLTR

Étape 1 : Identifier les processus

À partir du MCT ou du cahier des charges :

- Processus métier principaux
- Sous-processus si nécessaires

Étape 2 : Définir les opérations

Pour chaque processus :

- Opération déclenchée par un événement
- Opération produisant un résultat

Étape 3 : Définir les flux

- Quel message arrive ?
- Quel message sort ?
- Quel acteur les envoie ?

Étape 4 : Définir les acteurs

- Internes : systèmes, modules, services
- Externes : utilisateurs, partenaires

Étape 5 : Spécifier les règles

- Contraintes métier
- Conditions de déclenchement

Étape 6 : Construire le schéma MLTR

Un schéma clair avec :

- Blocs = processus
- Flèches = flux
- Découpage logique

Exemple d'application: Gestion de commandes

Processus identifiés :

- 1.Gérer Client
- 2.Gérer Commande
- 3.Gérer Stock

Exemple d'opérations

- Créer client
- Enregistrer commande
- Vérifier stock
- Diminuer stock

Acteurs

- Client
- Système Commande
- Admin

Flux principaux

- Demande de création de compte
- Demande commande
- Résultat Vérification stock
- Confirmation commande

Processus	Opération	Acteur	Flux entrant	Flux sortant	Règles de gestion
Gérer Client	Créer client	Client	Formulaire client	Confirmation création	Email unique
Gérer Commande	Enregistrer commande	Client	Demande commande	Confirmation commande	Client existant
Gérer Commande	Vérifier stock	Syst. Commande	ID produit	Résultat stock	Stock $\geq$ quantité
Gérer Stock	Diminuer stock	Syst. Commande	Commande validée	MAJ stock	Unité = 1 produit

Exemple:

Contexte : Gestion des emprunts d'une bibliothèque

Voici le **MCD complet**

LECTEUR (id_lecteur, nom, prenom, date_naissance, date_validite_carte)

LIVRE (id_livre, titre, auteur, categorie, stock)

EMPRUNT (id_emprunt, id_lecteur [FK], id_livre [FK])

À partir du MCD → Construction du MLTR

Nous allons extraire :

- ✓ Les processus
- ✓ Les opérations
- ✓ Les flux
- ✓ Les règles de gestion

A. Identification des processus (depuis les entités et gestion métier)

À partir du MCD, on retrouve trois grands traitements :

1.Gérer Lecteur

2.Gérer Livre

3.Gérer Emprunt

B. Identification des opérations

1. Gérer Lecteur

- Vérifier validité carte
- Rechercher lecteur

2. Gérer Livre

- Rechercher livre
- Vérifier disponibilité (stock > 0)

3. Gérer Emprunt

- Créer un emprunt
- Mettre à jour stock
- Enregistrer retour
- Mettre à jour stock
- Calculer pénalité (si retard)

C. Acteurs

- Lecteur
- Bibliothécaire
- Système automatisé

D. Règles de gestion (dédiées du MCD)

- RG1 : Un lecteur doit avoir **carte valide** ($\text{date_validite_carte} \geq \text{aujourd'hui}$)
- RG2 : Un livre ne peut être emprunté que si **stock** > 0

Processus	Opération	Acteur	Flux entrant	Flux sortant	Règle de gestion
Gérer Lecteur	Rechercher lecteur	Bibliothécaire	ID Lecteur	Fiche Lecteur	—
	Vérifier validité carte	Système	ID Lecteur	Résultat validité	RG1
Gérer Livre	Rechercher livre	Lecteur	Critère recherche	Liste livres	—
	Vérifier disponibilité	Système	ID livre	Résultat disponibilité	RG2
Gérer Emprunt	Créer emprunt	Bibliothécaire	Résultat vérifications	Fiche Emprunt	RG3
	Mettre à jour stock	Système	Emprunt validé	Nouveau stock	RG2

V-Modélisation de l'architecture du système informatique

1. Introduction

L'architecture du système informatique représente la manière dont les composants du SI sont organisés pour répondre aux besoins métiers. Elle permet de :

- Garantir la cohérence du système,
- Assurer la disponibilité et les performances,
- Maîtriser les évolutions,
- Renforcer la sécurité,
- Faciliter la maintenance et le déploiement.

La modélisation distingue généralement deux niveaux :

L'architecture logique :

une vision conceptuelle, indépendante des choix matériels ou logiciels.

L'architecture physique :

une vision concrète des technologies, serveurs, réseaux, etc.

Ce chapitre se concentre sur :

- **L'architecture logique du système**
- **L'architecture logique des moyens informatiques.**

2. Architecture Logique du Système

2.1. Définition

L'architecture logique du système est une **représentation fonctionnelle** du SI.

Elle décrit :

- Les **fonctions principales** du système,
- Les **modules** ou **sous-systèmes**,
- Les **flux d'information** entre ces services,
- Les **interactions** entre les acteurs et les traitements.

Elle ne dépend ni du matériel, ni des technologies utilisées.

2.2. Composants fonctionnels

Un système informatique est composé de plusieurs **sous-systèmes fonctionnels** comme :

- Gestion des utilisateurs
- Gestion du contenu
- Facturation
- Achats / ventes
- Analyse et reporting
- Authentification / autorisation
- Service client
- Gestion des fichiers et documents

Chaque sous-système est modélisé indépendamment des moyens techniques.

2.3. Modélisation des flux d'information

On identifie :

- Les **flux entrants** (clients, capteurs, utilisateurs internes),
- Les **flux sortants** (rapports, notifications, états),
- Les **flux internes** (entre modules et sous-systèmes).

Les modèles utilisés :

- Organigramme logique
- Diagrammes de flux (DFD – Data Flow Diagram)
- Matrices d'interaction entre modules

Exemple :

Le module *Commande* envoie un flux au module *Facturation* → Facture générée

2.4. Architecture logique applicative

On utilise généralement le principe du **découpage en couches logiques** :

• Couche Présentation

Communication avec l'utilisateur (interface web, mobile).

• Couche Métier (Business Logic)

Traitement des règles métier, validation, workflow, calculs.

• Couche Données

Gestion de la persistance des données, accès BD, fichiers, ORM.

- **Couche Communication / Services**

API REST, services internes, microservices.

Ce découpage permet :

- Modularité,
- Réutilisation,
- Maintenance facilitée,
- Evolutivité du système.

2.5. Outils et types de diagrammes

- **Diagramme de composants UML**

Montre les modules logiciels et leurs relations.

- **Diagramme d'interaction**

Circulation de messages et flux.

- **Diagramme de flux logique (DFD)**

Entrées → Traitements → Sorties.

- **Description fonctionnelle textuelle**

Spécification des rôles et responsabilités.

Exemple d'application:

Une **plateforme e-learning universitaire** permet aux étudiants et enseignants d'accéder à des services en ligne.

Les fonctionnalités principales sont :

- Authentification des utilisateurs
- Consultation des cours
- Dépôt des devoirs par les étudiants
- Correction et publication des notes par les enseignants
- Gestion des utilisateurs par l'administrateur

1. Identification des acteurs

Acteur	Rôle
Étudiant	Consulte cours, dépose devoir
Enseignant	Publie cours, corrige devoir
Administrateur	Gère utilisateurs
Système e-learning	Plateforme centrale

2. Fonctions principales

- S'authentifier
- Consulter les cours
- Déposer un devoir
- Corriger un devoir
- Publier les notes
- Gérer les utilisateurs

3. Diagrammes de l'architecture logique:

Diagramme de cas d'utilisation	
Étudiant	S'authentifier
Étudiant	Consulter cours
Étudiant	Déposer un devoir
Enseignant	S'authentifier
Enseignant	Publier cours
Enseignant	Corriger devoir)
Enseignant	Publier notes
Admin	Gérer utilisateurs

A. Diagramme de cas d'utilisation

Ce diagramme montre **ce que fait le système**, pas comment.

B- Diagramme de composants

Diagramme de composants
Module Gestion des cours
Module Gestion des devoirs
Module Gestion des notes
Module Administration

- Aucun serveur
- Aucun langage
- Aucun matériel

3. Architecture Logique des Moyens Informatiques

3.1. Définition

L'architecture logique des moyens informatiques décrit **l'organisation conceptuelle** des ressources techniques nécessaires au fonctionnement du système :

- Serveurs,
- Bases de données,
- Services principaux,
- Réseaux logiques,
- Sécurité logique.

Elle reste **indépendante du matériel exact** (marque, modèle, dimensionnement).

3.2. Moyens informatiques (vision logique)

- **Serveurs logiques**
 - Serveur d'authentification (AD, LDAP)
 - Serveur applicatif
 - Serveur web
 - Serveur de base de données
 - Serveur de sauvegarde
 - Serveur de messagerie

- **Stockage logique**

- Base opérationnelle (OLTP)
- Entrepôt de données (Data Warehouse)
- Stockage documentaire (NAS, Cloud Storage)

- **Réseau logique**

- VLAN par fonction
- Segments réseau : utilisateurs, serveurs, DMZ
- Domaines logiques

- **Services logiques transversaux**

- DNS
- DHCP
- Répartition de charge (Load Balancer)
- Services proxy et pare-feu logiques
- Services cryptographiques (PKI)

3.3. Modèle logique d'infrastructure

Exemple de découpage logique :

- Serveur Applicatif (héberge les modules métier)
- Serveur Web (IHM, API)
- Serveur BD (données)
- Serveur Sécurité (authentification)
- Serveur Backup (sauvegarde quotidienne)

Chaque élément est décrit par :

- Ses rôles,
- Ses interactions logiques,
- Ses flux autorisés.

3.4. Architecture logique du réseau

On modélise :

- **Zones logiques**
 - **Zone interne** (accès employés)
 - **Zone DMZ** (exposition publique)
 - **Zone stockage**
 - **Zone administration**

- **Flux réseau autorisés**

Exemple :

Le Serveur Web peut communiquer avec le Serveur Applicatif → Oui

Le Serveur Web peut communiquer avec le Serveur BD → Non (sécurité)

- **Sous-réseaux logiques**

- VLAN utilisateurs
- VLAN serveurs
- VLAN sécurité

3.5. Rôles et interactions

On décrit comment chaque composant logique interagit :

- Serveur Web appelle API du serveur applicatif
- Serveur applicatif interroge la base de données
- Serveur d'authentification vérifie les identités
- Serveur de sauvegarde récupère les données du serveur BD

Ces interactions logiques assurent :

- La performance
- La sécurité
- La cohérence
- La continuité d'activité

4. Lien entre les deux architectures

Architecture logique du système	Architecture logique des moyens
Vue fonctionnelle	Vue technique logique
Modules / services	Serveurs / BD / réseau
Flux fonctionnels	Flux réseau
Dépend d'une analyse métier	Dépend de l'infrastructure

Exemple : Système de Gestion d'École (SGE)

L'école veut un système informatique pour gérer :

- Les étudiants
- Les enseignants
- Les notes
- Les inscriptions

1. Objectif de l'architecture logique des moyens: définir comment organiser les ressources informatiques logiquement, pour supporter le système, sans entrer dans le matériel physique.

2. Les Serveurs Logiques (rôles)

Dans ce système scolaire, les besoins logiques sont :

Serveur Web

Affiche l'interface élève / enseignant.

- Portail Étudiant
- Portail Enseignant

Serveur Applicatif

Contient les règles métiers :

- Calcul des moyennes
- Validations d'inscription
- Gestion des notes

Serveur Base de Données (BD)

Stocke :

- Etudiants
- Enseignants
- Modules
- Notes
- Historiques

Serveur d'Authentification

Gère la connexion :

- LDAP / Active Directory logique
- Gestion des rôles

Serveur de Sauvegarde (logiciel de backup)

Permet :

- Sauvegarde quotidienne
- Restauration en cas de problème

Ce ne sont pas des machines réelles, juste des rôles logiques.

3. Les Zones Réseau Logiques

DMZ (zone publique)

- Portail web étudiant
- Portail web enseignant

Accessible depuis Internet (logiquement).

Réseau Interne (zone privée)

- Serveur applicatif
- Serveur base de données
- Serveur d'authentification



Invisible depuis l'extérieur.

Réseau Admin

- Serveur de sauvegarde
- Outils d'administration IT

4. Les Flux Logiques entre les composants

1. Le **serveur web** envoie les demandes au **serveur applicatif**.
2. Le **serveur applicatif** interroge la **base de données**.
3. Le **serveur applicatif** vérifie les utilisateurs auprès du **serveur d'authentification**.
4. Le **serveur de sauvegarde** récupère chaque jour les données de la BD.



Tout cela est logique, sans parler d'IP ou de matériel.

Exemple d'application

On considère une **application web universitaire sécurisée** permettant :

- Aux étudiants de consulter leurs notes,
- Aux enseignants de saisir les notes,
- A l'administration de gérer les utilisateurs.

1. Identification des moyens informatiques (logiques)

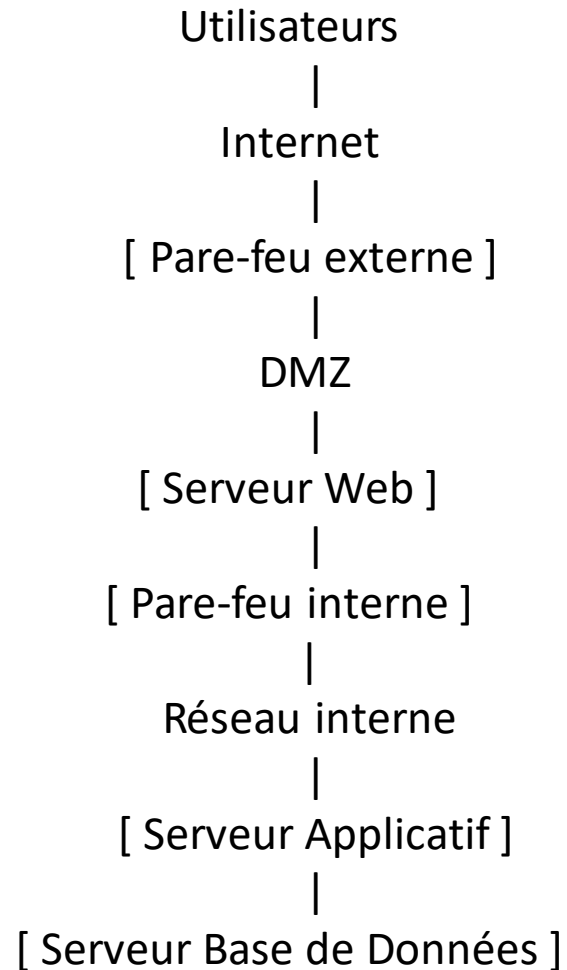
Acteurs techniques

- Utilisateurs (navigateurs web)
- Réseau Internet

Ressources IT (logiques)

- Pare-feu
- Zone DMZ
- Serveur Web
- Serveur Applicatif
- Serveur Base de Données

2. Architecture réseau logique de l'application



- Tous les éléments sont **logiques**
- Aucun matériel réel
- Aucun IP

3. Rôle logique de chaque composant

Composant	Rôle
Pare-feu	Filtrage des flux
DMZ	Zone exposée
Serveur Web	Interface utilisateur
Serveur Applicatif	Logique métier
Serveur BD	Stockage sécurisé

VI- Le langage UML

UML (Unified Modeling Language)

I. Introduction à UML (Unified Modeling Language)

1. Définition et rôle de UML

UML est un **langage de modélisation standardisé** qui permet de représenter graphiquement un système, ses composants et leur interaction. Il n'est pas un langage de programmation, mais un **outil de communication et de documentation** pour les analystes, développeurs et clients.

UML sert à :

- Comprendre et formaliser les besoins d'un système.
- Concevoir des architectures logicielles cohérentes.
- Faciliter la maintenance et l'évolution du système

2. Historique et origines

Dans les années 1990, plusieurs méthodes de modélisation orientée objet existaient : **Booch**, **OMT (Object Modeling Technique)**, **OOSE (Object-Oriented Software Engineering)**.

En 1997, l'**OMG (Object Management Group)** a standardisé ces approches sous le nom **UML**.

Depuis, UML est devenu le **standard international** pour la modélisation de systèmes orientés objet, utilisé dans l'industrie et l'enseignement

3. Objectifs de UML

- **Communication** : faciliter l'échange d'informations entre toutes les parties prenantes (analystes, développeurs, clients, chefs de projet).
- **Documentation** : conserver une trace claire des décisions de conception.
- **Standardisation** : permettre à différents acteurs et équipes de parler le même « langage » technique.
- **Conception orientée objet** : représenter classes, objets, relations, comportements.
- **Support à la qualité logicielle** : identifier les problèmes tôt et éviter les erreurs coûteuses en phase de développement.

4. Domaines d'application de UML

- Développement logiciel (applications web, mobile, systèmes embarqués...)
- Systèmes d'information (gestion d'entreprise, hôpital, banque...)
- Ingénierie des systèmes complexes (industrie, transport, énergie...)
- Modélisation de processus métiers et analyse organisationnelle

5. Structure générale de UML

UML propose plusieurs **types de diagrammes** regroupés en deux catégories principales :

- **Diagrammes structurels** : classes, objets, composants, déploiement, packages.
- **Diagrammes comportementaux** : cas d'utilisation, séquence, activités, états.

Chaque diagramme a une **finalité spécifique**, mais tous contribuent à une **vision globale cohérente** du système

II. Concepts fondamentaux de UML

1. Objets et classes

Classe : modèle abstrait représentant un ensemble d'objets ayant les mêmes caractéristiques et comportements.

Exemple : Patient dans un hôpital, CompteBancaire dans une banque.

- **Composants d'une classe** : Nom de la classe : identifiant de l'entité (Patient) Attributs : propriétés ou caractéristiques (*nom*, *dateNaissance*)
- **Méthodes (opérations)** : actions que la classe peut réaliser (*payerFacture()*, *prendreRDV()*)
- **Objet** : instance concrète d'une classe.
- Exemple : Patient1 = {*nom*: "Ali", *dateNaissance*: "1990-05-12"}

2. Relations entre classes

UML permet de représenter différents types de liens entre classes :

Association

1. Relation structurelle simple entre deux classes.
2. Peut avoir une **multiplicité** (1..1, 0..*, etc.)

Héritage (Généralisation)

- Une classe enfant hérite des attributs et méthodes d'une classe parent.
- Représenté par une flèche vide pointant vers la super-classe.

Agrégation:

- Relation « tout/partie » faible.
- La partie peut exister indépendamment du tout.
- **Exemple** : Département contient plusieurs Médecins

Composition:

- Relation « tout/partie » forte.
- La partie ne peut exister sans le tout.
- **Exemple** : *DossierPatient* contient *Ordonnance* (l'ordonnance n'existe pas sans le dossier)

4. Diagrammes UML : structurels et comportementaux

Diagrammes structurels : montrent la structure du système.

- Diagrammes de classes, d'objets, de composants, de déploiement.

Diagrammes comportementaux : montrent comment le système se comporte dans le temps.

- Diagrammes de cas d'utilisation, de séquence, d'activités, d'états.

III : Les diagrammes structurels

Les diagrammes structurels décrivent la **structure statique** d'un système : classes, objets, composants, nœuds de déploiement...

Ils montrent *ce que le système contient*, indépendamment de son comportement dynamique.

UML comprend 5 principaux diagrammes structurels :

1. Diagramme de classes

C'est le diagramme le plus utilisé en UML.

Il représente :

- Les **classes**
- Leurs **attributs** et **méthodes**
- Les **relations** entre classes (association, héritage, composition, agrégation)
- Les **multiplicités**

Exemple : Système de gestion d'hôpital

Classes :

Patient (nom, âge, numéroDossier)

Médecin (nom, spécialité)

Consultation (date, diagnostic)

IV. Les Diagrammes Comportementaux UML

Les diagrammes comportementaux décrivent **la dynamique du système**, c'est-à-dire *comment il se comporte, réagit, interagit et évolue dans le temps*.

Ils répondent à la question :

Que fait le système ?"

1. Diagramme de Cas d'Usage (Use Case Diagram)

Représenter les *interactions entre les acteurs* (utilisateurs ou systèmes externes) et les *fonctionnalités principales* du système.

Éléments clés:

- Acteurs
- Cas d'usage (use cases)
- Relations : include, extend, généralisation
- Limite du système

Quand l'utiliser ?

- Analyse des besoins
- Communication avec le client
- Définition du périmètre fonctionnel

IV. Les Diagrammes Comportementaux UML

1. Diagramme de Séquence (Sequence Diagram)

Décrire les **interactions temporelles** entre objets, sous forme de messages échangés.

Éléments clés:

- Lifelines (lignes de vie)
- Messages synchrones/asynchrones
- Activation
- Retour
- Boucles, alternatives, conditions

Quand l'utiliser ?

- Décrire un scénario d'utilisation
- Conception détaillée
- Communication entre objets dans le temps

Résumé des types de diagrammes UML

1) Diagrammes Structurels (STRUCTURE)

Ils décrivent **l'architecture statique** du système : les classes, les objets, les composants, l'infrastructure...

Ils répondent à :

Diagrammes structurels les plus utilisés :

1. **Diagramme de Classes:** Structure du système (classes, attributs, relations)
2. **Diagramme d'Objets:** Instances concrètes des classes
3. **Diagramme de Composants:** Architecture logicielle modulaire (modules, interfaces)
4. **Diagramme de Déploiement:** Architecture physique (serveurs, réseau, BD, client)

Les autres diagrammes structurels (moins utilisés) :

- Diagramme de Packages
- Diagramme de Structure Composite
- Diagramme de Profil

2) Diagrammes Comportementaux (COMPORTEMENT)

Ils décrivent **la dynamique** : ce que fait le système, comment il réagit, comment les objets interagissent.

Ils répondent à : **Que fait le système ? & Comment les acteurs interagissent ?**

Diagrammes comportementaux les plus utilisés :

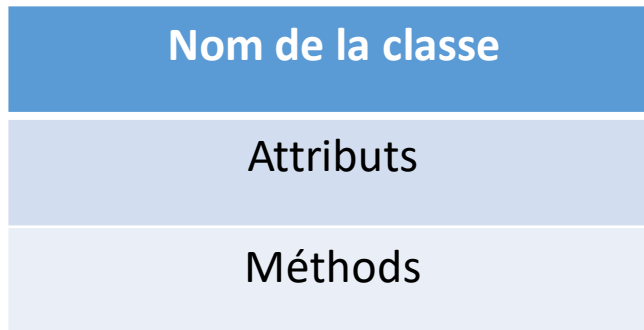
1. **Diagramme de Cas d'Usage (Use Case):** Interactions entre acteurs et système
2. **Diagramme d'Activité:** Workflow, processus métier
3. **Diagramme de Séquence:** Interaction temporelle entre objets
4. **Diagramme d'État:** Cycle de vie d'un objet

Autres diagrammes comportementaux :

- Diagramme de Communication
- Diagramme de Timing
- Diagramme d'Interaction Globale

Comment présenter graphiquement un diagramme de classes UML

Un **diagramme de classes** doit respecter une structure graphique standard.
Chaque classe se représente dans un **rectangle divisé en 3 parties** :



Légende des symboles UML :

- + public
- – private
- # protected
- *italique* = abstrait

Les relations se présentent sous forme de **lignes annotées**, avec parfois des symboles.

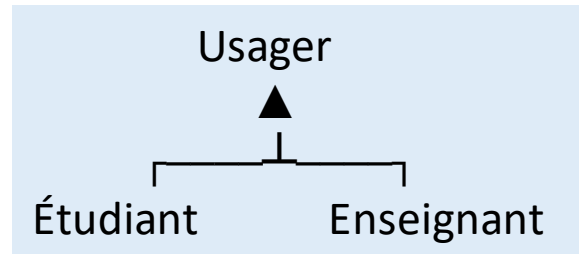
Association simple: Livre ----- possède -----> Exempleire

Multiplicités (cardinalités):

Livre Exempleire
 1 ----- 1..*

Héritage (généralisation)

Triangle vide pointant vers la classe mère :



Composition (remplie)

Le losange noir signifie : "fait partie de".

Bibliothèque ◆— 1..* Livre

Agrégation (losange blanc):

Commande ◇— 1..* LigneCommande

Présentation graphique d'un diagramme de cas d'utilisation

Encadrer le système

- Dessiner un **rectangle** pour représenter le système étudié.
- Tous les **cas d'utilisation** se trouvent à **l'intérieur** du rectangle.
- **Exemple** : rectangle intitulé « Système bancaire ».

Placer les acteurs

- Les acteurs sont à **l'extérieur** du rectangle.
- **Graphiquement** : stickman (bonhomme bâton).
- Ils peuvent être placés à **gauche, droite ou en haut** du diagramme selon la clarté.

Dessiner les cas d'utilisation

- Chaque fonction ou service du système est un **cas d'utilisation**.
- **Graphiquement** : ovale (ellipse) avec le nom de la fonction à l'intérieur.
- Exemples : « Se connecter », « Consulter solde », « Effectuer virement ».

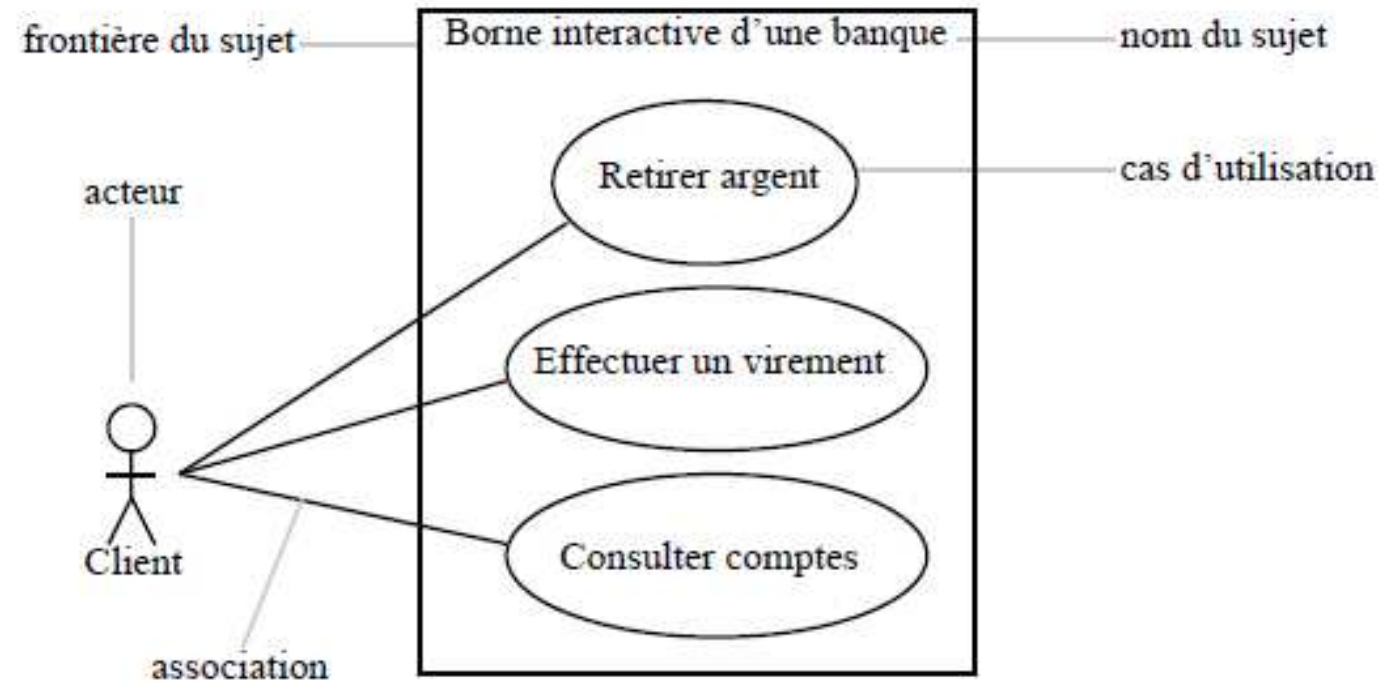
Relier acteurs et cas

- Une **ligne simple** montre l'interaction entre un acteur et un cas d'utilisation.
- Exemple : l'acteur « Utilisateur » relié à « Se connecter » et « Consulter solde ».

Ajouter les relations entre cas

- **Include (<<include>>)** : un cas est **toujours exécuté** par un autre.
 - Graphiquement : flèche pointillée avec <<include>>.
- **Extend (<<extend>>)** : un cas est **optionnel ou conditionnel**.
 - Graphiquement : flèche pointillée avec <<extend>>.
- **Généralisation** : un acteur ou un cas peut être une **spécialisation** d'un autre.
 - Graphiquement : flèche avec triangle vide.

Exemple:



Présentation graphique d'un diagramme de séquence

a) Acteurs et objets

- Placés horizontalement en haut du diagramme.
- Chaque acteur/objet possède une **ligne de vie (lifeline)** représentée par une ligne verticale.

b) Messages

- Les messages sont représentés par des flèches horizontales.
- Types :
 - **Message synchrone** : demande de traitement (flèche pleine).
 - **Message asynchrone** : envoi sans attendre une réponse.
 - **Message de retour** : flèche pointillée indiquant la réponse.

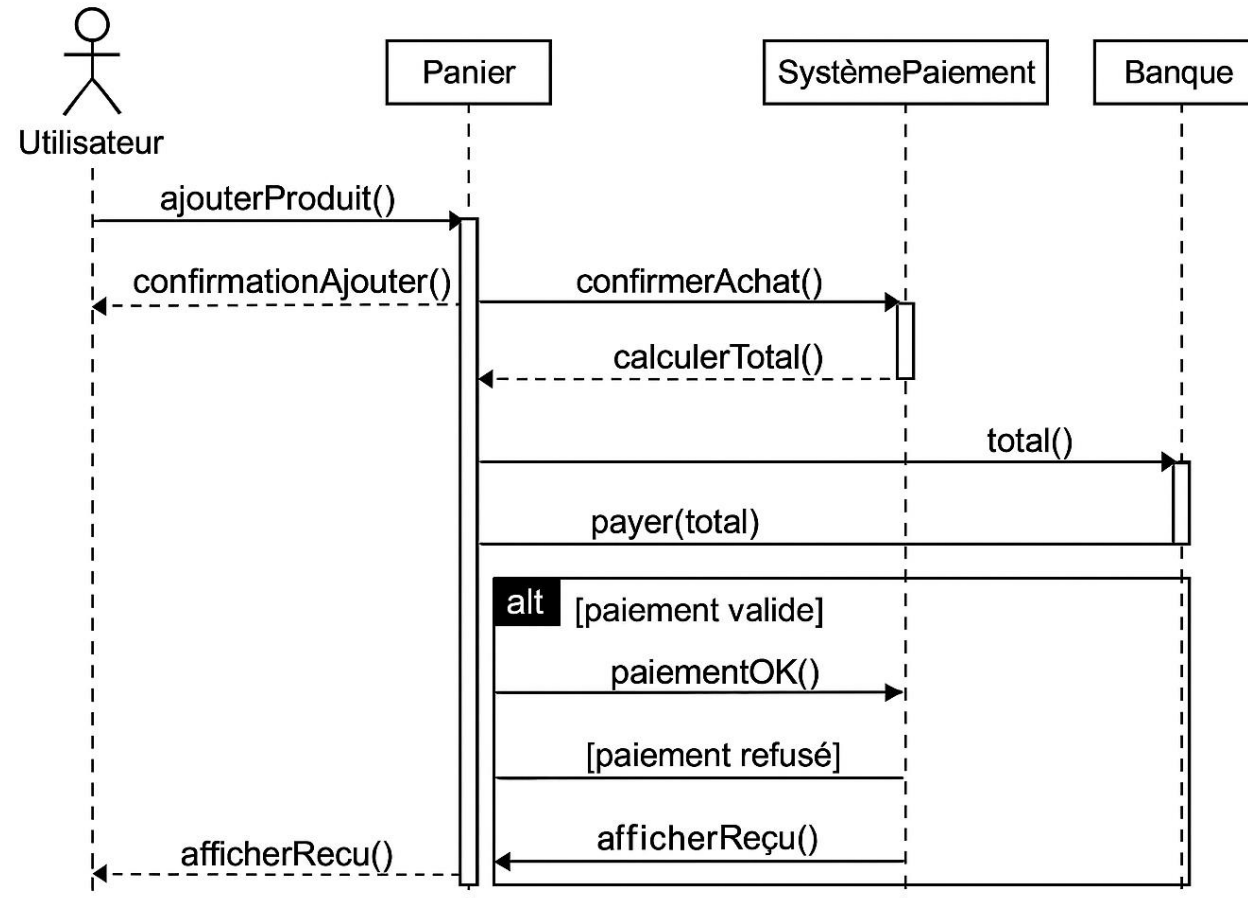
c) Activation

- Petit rectangle vertical sur une ligne de vie.
- Représente la période où un objet exécute une opération.

d) Fragments combinés

- Utilisés pour représenter la logique :
 - **alt** : alternatives (si/sinon)
 - **loop** : répétitions
 - **opt** : optionnel

Exemple:



Exercice :

Vous devez modéliser un **système de gestion d'une bibliothèque**, le cahier des charges suivant décrit les besoins :

- Une **Bibliothèque** possède plusieurs **Livres**.
- Lorsqu'une bibliothèque ferme définitivement, les livres sont supprimés
- Chaque **Livre** possède : un titre, un auteur, un ISBN, un état (disponible/emprunté).
- Un **Adhérent** peut emprunter plusieurs livres, mais un livre ne peut être emprunté que par **un seul adhérent à la fois**.
- Un **Emprunt** permet d'enregistrer la date d'emprunt et de retour prévue.
- Un emprunt est lié à **un seul adhérent** et **un seul livre**.
- Un adhérent possède : nom, prénom, numéro-carte.

Travail demandé:

À partir du cahier des charges :

- Déterminer les **classes nécessaires**
- Définir les **attributs**
- Définir les **relations UML** (composition, association)
- Définir les **multiplicités**
- Produire un **diagramme de classes UML complet**

Correction complète

Classe	Attributs
Bibliothèque	nom
Livre	titre, auteur, ISBN, état
Adhérent	nom, prénom, numCarte
Emprunt	dateEmprunt, dateRetourPrevue

2. Relations UML

a) Bibliothèque — Livre

- Une bibliothèque compose plusieurs livres
- **Composition** (losange noir)
- Multiplicité :
 - Bibliothèque 1 → Livre 0..*
 - Livre 1 → Bibliothèque 1

b) Adhérent — Livre

- Un adhérent peut emprunter plusieurs livres
- Un livre ne peut être emprunté que par un seul adhérent à la fois
- Multiplicité : Adhérent 1..* → Livre 0..1

Adhérent — Emprunt — Livre

- Un emprunt lie un adhérent et un livre
- Multiplicités :
 - Adhérent 1 → Emprunt 0..*
 - Livre 1 → Emprunt 0..*

UML:

